

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and communications technology

Software Design Document

AIMS

Subject: Software Design and Construction

Group 3

Participants:

Nguyen Trung Chien 20215320

Nguyen Manh Cuong 20204524

Vu Ngoc Anh 20215313

Bui Minh Hai Dac 20204525

Tran Duc Chinh 20210123

Hanoi, 12/2024

Table of Contents

Table of Contents	1
1 Introduction	7
1.1 Objective.....	7
1.2 Scope	7
1.3 Glossary	7
1.4 References	7
2 Overall Description	8
2.1 General Overview	8
2.2 Assumptions/Constraints/Risks	9
2.2.1 Assumptions.....	9
2.2.2 Constraints	10
2.2.3 Risks.....	11
3 System Architecture and Architecture Design	12
3.1 Architectural Patterns	12
3.2 Interaction Diagrams	12
3.3 Analysis Class Diagrams	15
3.4 Unified Analysis Class Diagram	17
3.5 Security Software Architecture	17
4 Detailed Design	23
4.1 User Interface Design	23
4.1.1 Screen Configuration Standardization	23
4.1.2 Screen Transition Diagrams.....	24
4.1.3 Screen Specifications	24
4.2 Data Modeling	36
4.2.1 Conceptual Data Modeling	36
4.2.2 Database Design.....	36
4.3 Non-Database Management System Files	55

4.4	Class Design	55
4.4.1	General Class Diagram	55
4.4.2	Class Diagrams	55
4.4.3	Class Design.....	58
5	Design Considerations.....	77
5.1	Goals and Guidelines.....	77
5.2	Architectural Strategies	77
5.3	Coupling and Cohesion	79
5.4	Design Principles.....	80
5.5	Design Patterns	81
5.5.1	Singleton Pattern.....	81
5.5.2	Dependency Injection	82
5.5.3	Factory Method Pattern.....	82
5.5.4	Template Method Pattern.....	82
5.5.5	Adapter Pattern	83
5.5.6	Builder Pattern	84
5.6	Strategy Pattern.....	84

List of Figures

Figure 1 General usecase diagram	9
Figure 2 Place order interaction diagram.....	13
Figure 3 Pay order interaction diagram	14
Figure 4 Place order analysis class diagram	15
Figure 5 Pay order analysis class diagram.....	16
Figure 6 Unified analysis class diagram	17
Figure 7 Screen transition diagram	24
Figure 8 Database design	38
Figure 9 General class diagram.....	55
Figure 10 Strategy pattern in payment handling.....	84

List of Tables

Table 1 Home screen specifications	24
Table 2 Cart screen specifications	25
Table 3 Delivery screen specifications	27
Table 4 Invoice screen specifications	28
Table 5 Payment screen specifications	29
Table 6 Successful order screen specifications.....	30
Table 7 Home screen attributes.....	32
Table 8 Cart screen attributes	32
Table 9 Delivery screen attributes	33
Table 10 Invoice screen attributes	33
Table 11 Payment screen attributes	34
Table 12 Successful order screen attributes.....	35
Table 13 Conceptual ERD	36
Table 14. User table	38
Table 15 Product table	39
Table 16 Product_media_urls table	40
Table 17 Order table	40
Table 18 Product_cart table	40
Table 19 Invoice table.....	41
Table 20 Order_product table	41
Table 21 Transaction_info table	42
Table 22 Favorite_product_user table	42
Table 23 Book table	43
Table 24 CD table	43
Table 25 CD_tracklist table	44
Table 26 DVD table	44
Table 27 Review table	45

Table 28 Review_media_urls	45
Table 29 Class design AuthController	58
Table 30 Class design CartController	58
Table 31 Class design OrderController	59
Table 32 Class design PaymentController	59
Table 33 Class design ProductController	60
Table 34 Class design UserController	60
Table 35 Class design BaseEntity	61
Table 36 Class design Book	61
Table 37 Class design CD	62
Table 38 Class design DVD	62
Table 39 Class design FavoriteProductUser	63
Table 40 Class design Invoice	63
Table 41 Class design Order	64
Table 42 Class design OrderProduct	65
Table 43 Class design Product	65
Table 44 Class design ProductCart	66
Table 45 Class design TransactionInfo	66
Table 46 Class design User	67
Table 47 Class design CartService	67
Table 48 Class design EmailService	67
Table 49 Class design JwtService	68
Table 50 Class design MediaService	68
Table 51 Class design EmailService	68
Table 52 Class design UserService	69
Table 53 Class design ProductService	70
Table 54 Class design PaymentService	71
Table 55 Class design OtpService	72
Table 56 Class design OrderService	73

Table 57 Class design MediaService	74
Table 58 Class design JwtService	75
Table 59 Common coupling improvements.....	79
Table 60 Stamp coupling improvements	80

1 Introduction

1.1 Objective

This document is the specification of an internet media store, and is tailored for designers, developers and operators to execute this project. Moreover, technical executives can have a grasp of the project for monitoring purposes and users with hope to thoroughly understand the system can also acquire knowledge of the software.

1.2 Scope

Being analyzed in this document is a software called AIMS, stands for An Internet Media Store. As the name suggests, it provides products owners tools to set up and manage their shop online, enables people to buy the merchandise. In executives' point of view, the software is taken advantage as a management tool whose targets are online customers and product manager.

1.3 Glossary

<i>No</i>	<i>Term</i>	<i>Explanation</i>	<i>Example</i>	<i>Note</i>
1	cart	A list of products that the customers want to buy	[CD Lover (1), DVD The on1y one (1)....]	Names are usually followed by quantity

1.4 References

AIMS-problemStatement-v2.0 by Prof. Nguyen Thi Thu Trang

2 Overall Description

2.1 General Overview

AIMS (An Internet Media Store) is a desktop e-commerce application designed for selling physical media products including books, CDs, LP records, and DVDs. The system follows a client-server architecture with a focus on providing a robust, high-performance shopping experience.

The primary design goals of the system include:

- High availability (24/7 operation with 1-hour maximum recovery time)
- Scalability (supporting up to 1,000 concurrent users)
- Performance optimization (2-second response time under normal conditions, 5 seconds during peak hours)
- System reliability (300 hours continuous operation without failure)

The system is organized around three main user roles:

- Administrators - Managing user accounts and system access
- Product Managers - Handling product inventory and order management
- Customers - Browsing products, managing shopping carts, and placing orders

The system integrates with external services, notably VNPay for payment processing, and implements an email notification system for order management and administrative actions.

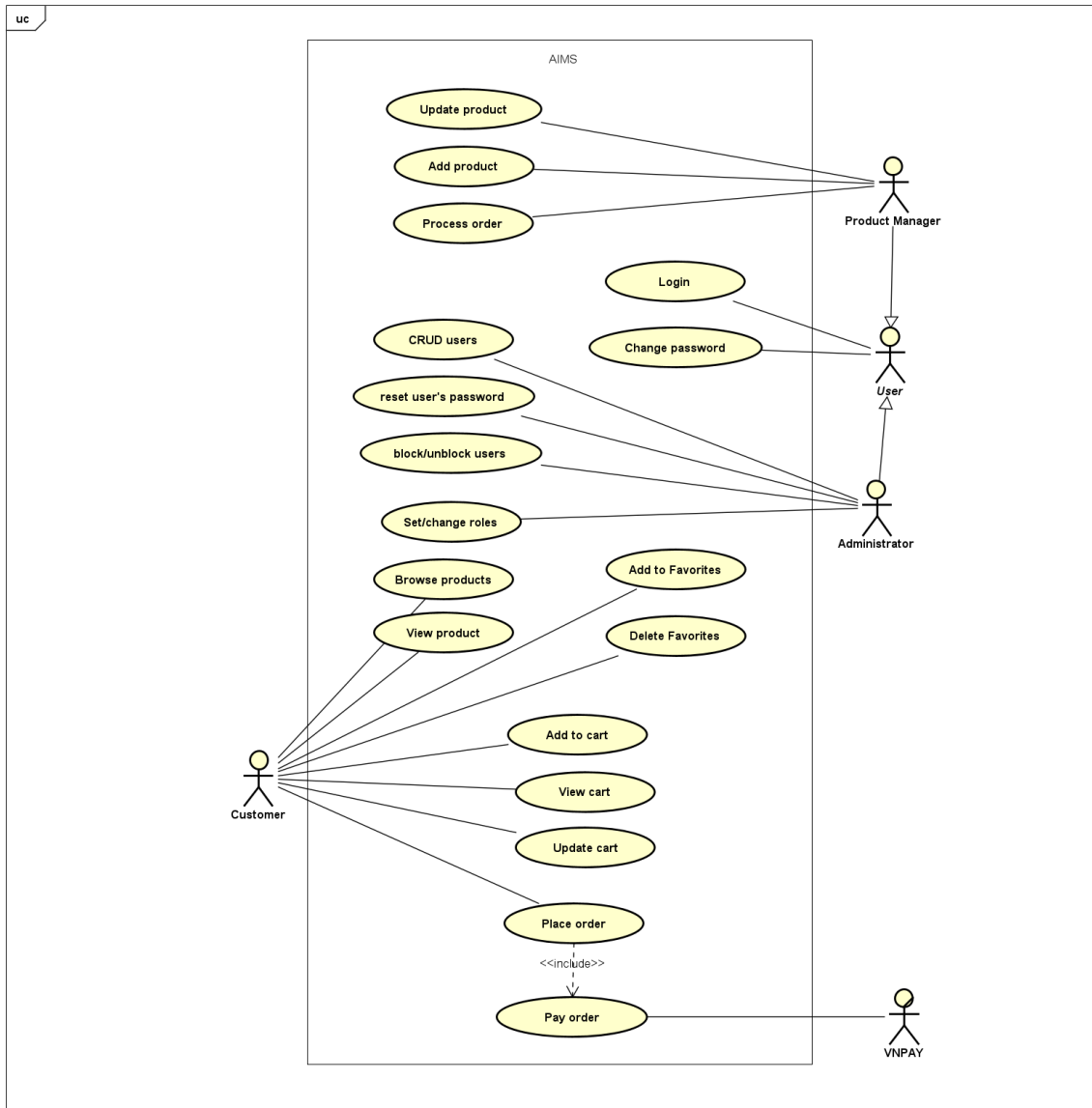


Figure 1 General usecase diagram

2.2 Assumptions/Constraints/Risks

2.2.1 Assumptions

Operating Environment

- Users have access to desktop computers, having browser capable of running the application
- Stable internet connectivity is available for payment processing and email notifications
- VNPay Sandbox environment is available for testing payment transactions

User Characteristics

- Product managers are trained in inventory management procedures
- Customers can understand basic e-commerce interfaces
- Users have basic computer literacy and understanding of online shopping processes

System Usage

- Peak usage periods will not exceed the designed 1,000 concurrent user limit
- Product catalog will primarily consist of physical media items
- Email notifications will be deliverable to users

2.2.2 Constraints

Hardware/Software Environment

- Desktop-only application platform
- Must maintain connectivity with VNPay payment gateway
- Must support email notification functionality

Performance Requirements

- Maximum response time of 2 seconds under normal conditions
- Maximum response time of 5 seconds during peak hours
- Support for 1,000 concurrent users
- 300-hour continuous operation capability
- 1-hour maximum recovery time after incidents

Security and Access Control

- Product managers limited to 30 product updates/deletions per day
- Maximum 10 simultaneous product deletions
- Two price updates per product per day
- Product pricing restricted to 30-150% of product value

Business Rules

- Rush delivery limited to Hanoi inner city districts
- VAT fixed at 10%
- Specific shipping fee structure based on weight and location
- Free shipping rules for orders over 100,000 VND

Data Management

- Single cart per session

- Display limit of 20 products per page
- Display of 30 pending orders per page for product managers

2.2.3 Risks

Technical Risks

- System performance degradation during peak hours
- Payment gateway integration failures
- Email notification system failures
- Database performance issues with large product catalogs

Business Risks

- Inventory synchronization issues during concurrent operations
- Order processing delays during high-volume periods
- Rush delivery service constraints in specific locations

Mitigation Strategies

- Implement robust error handling and system monitoring
- Regular system backups and failover procedures
- Clear user notification systems for service interruptions
- Performance optimization for database operations
- Comprehensive testing of payment gateway integration
- Regular system maintenance during off-peak hours

3 System Architecture and Architecture Design

3.1 Architectural Patterns

The AIMS system would benefit from the MVC pattern to separate concerns between data management, user interface, and business logic:

- Model: Handles product data, user information, orders, and cart management
- View: Manages multiple user interfaces for different roles (admin, product manager, customer)
- Controller: Processes user actions and business rules (e.g., price validation, rush delivery eligibility)

Rationale: MVC provides clear separation of concerns, making the system easier to maintain and modify, especially important given the different user roles and complex business rules.

3.2 Interaction Diagrams

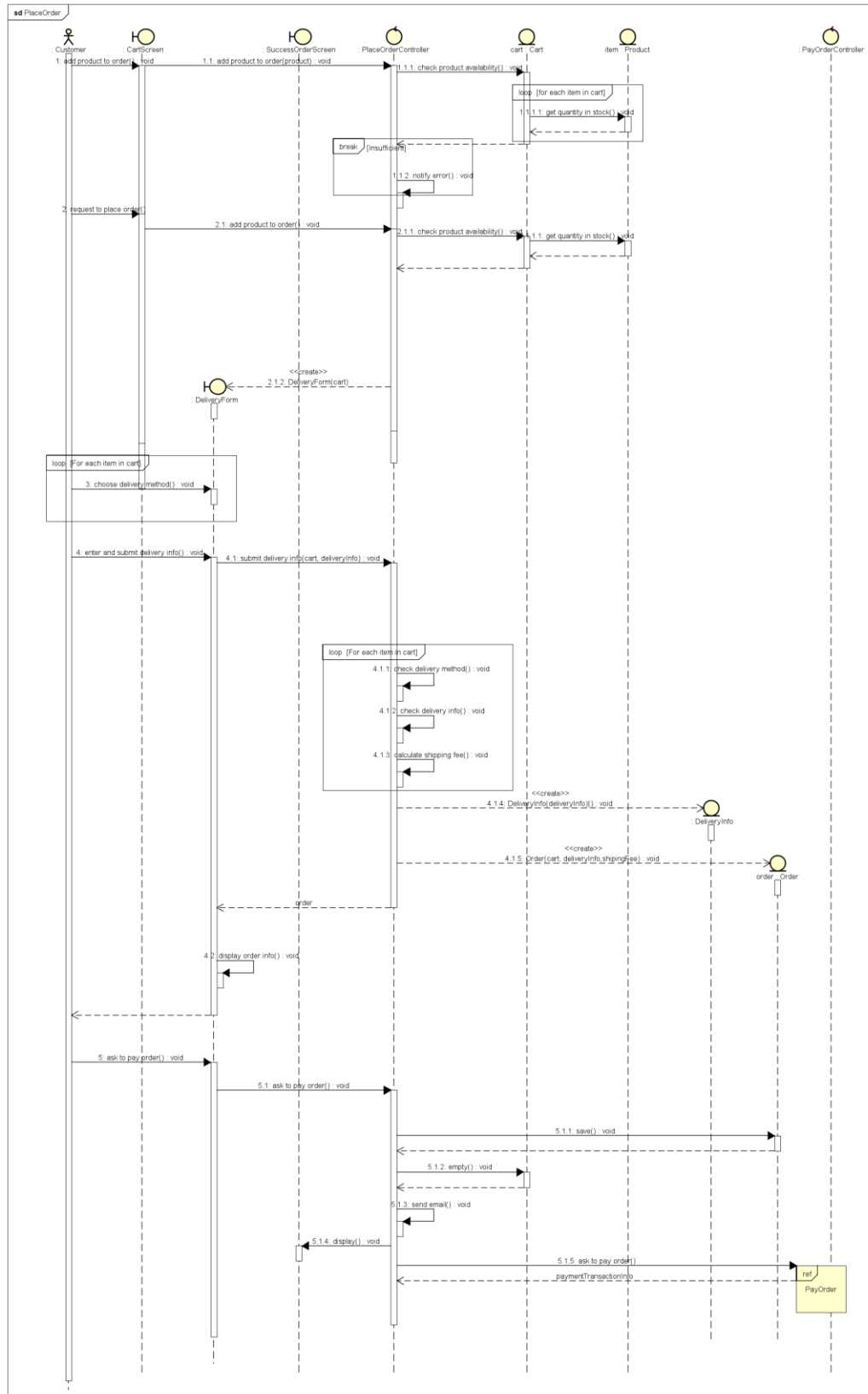


Figure 2 Place order interaction diagram

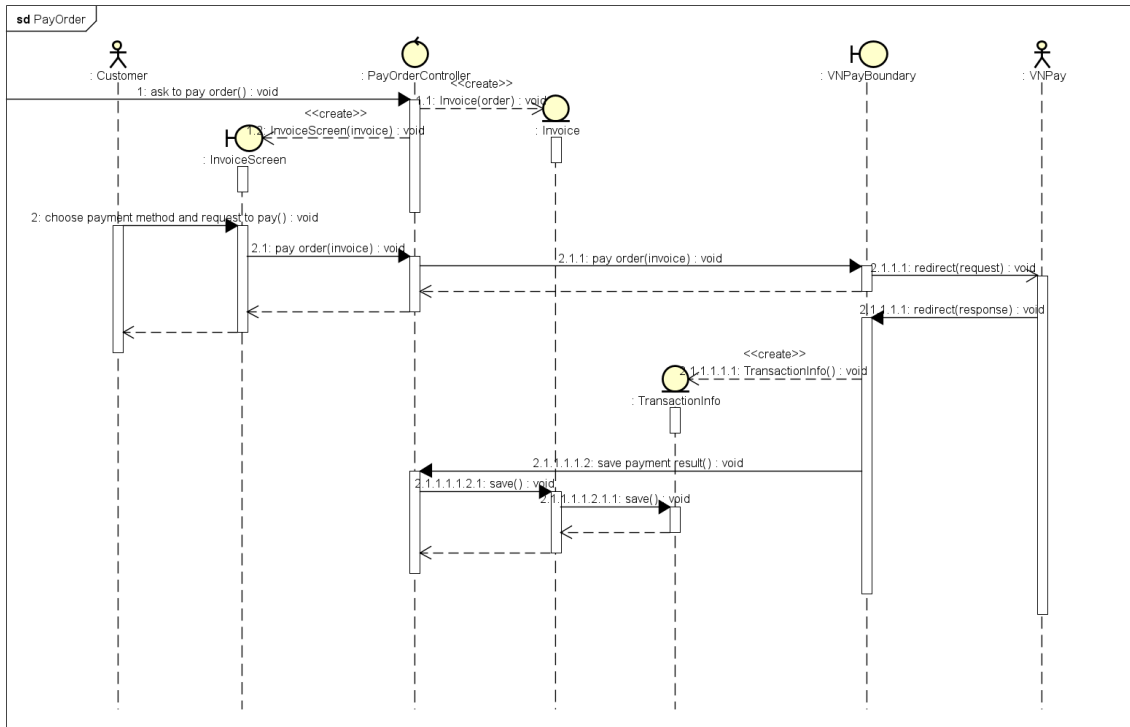


Figure 3 Pay order interaction diagram

3.3 Analysis Class Diagrams

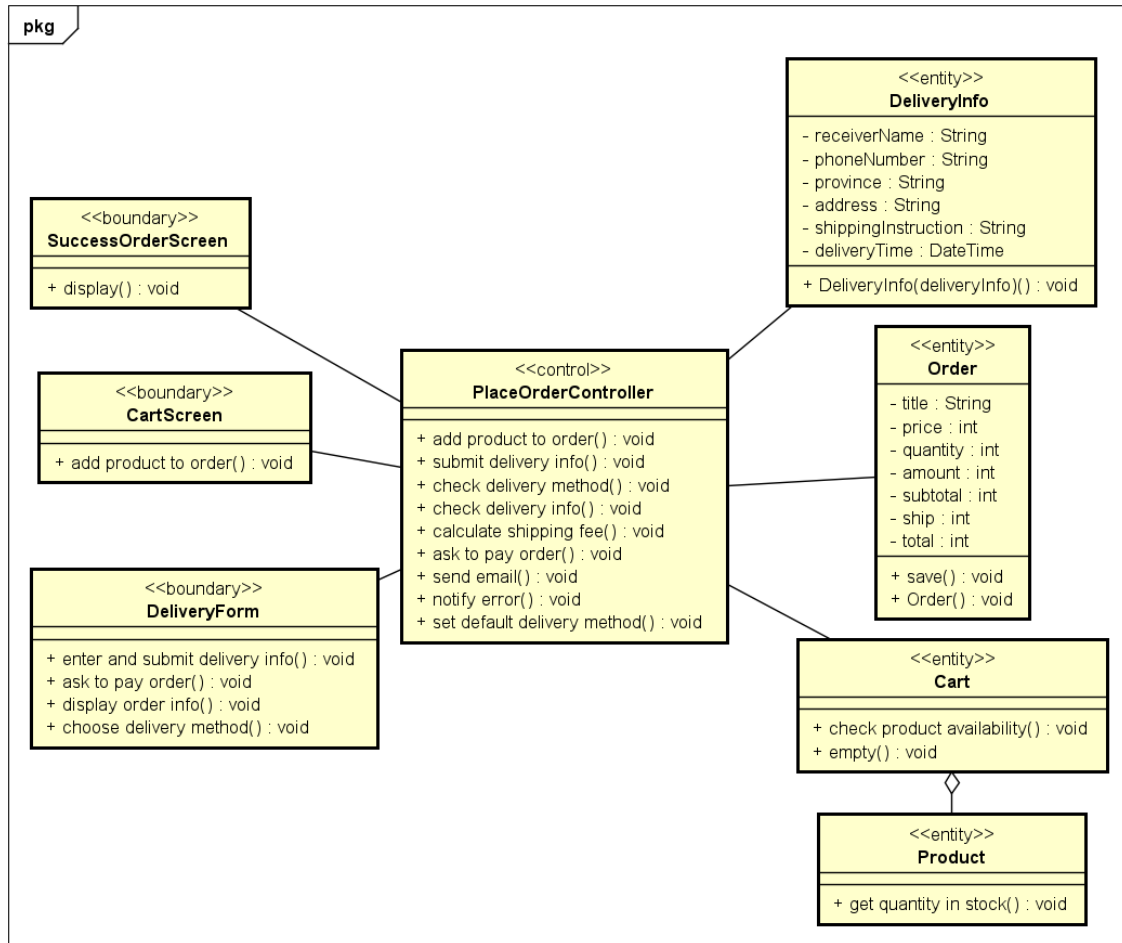


Figure 4 Place order analysis class diagram

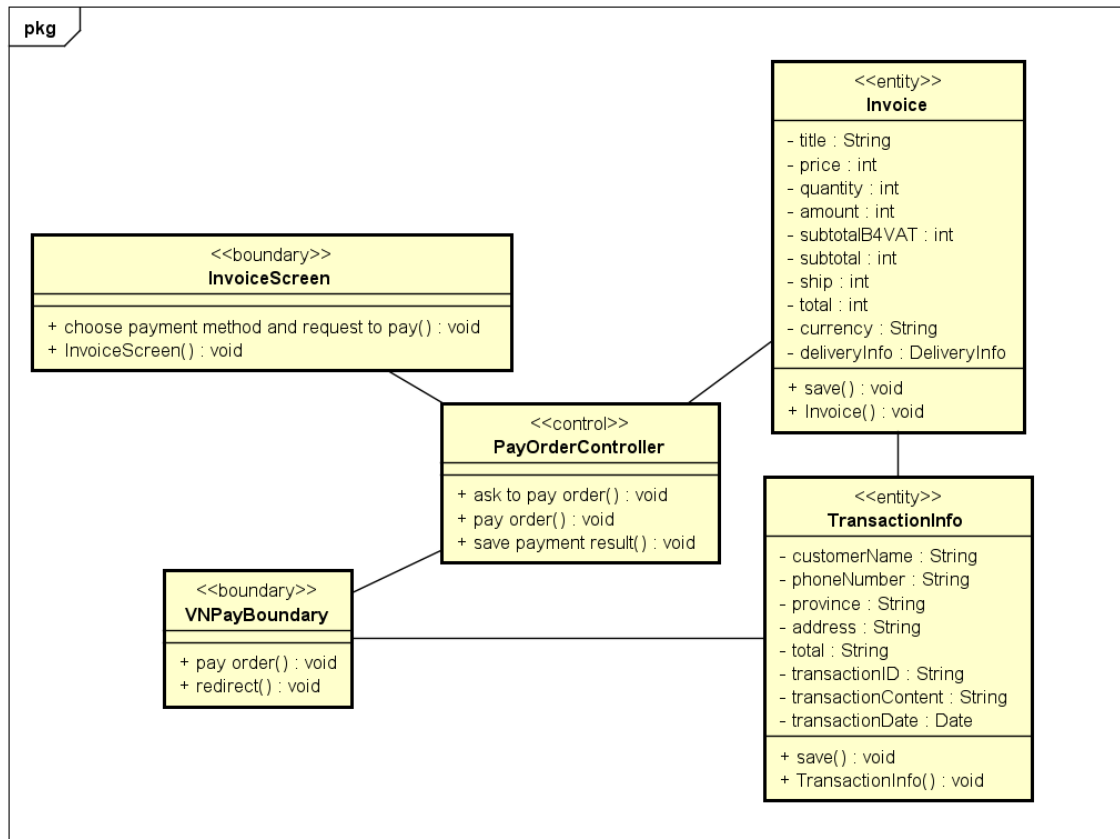


Figure 5 Pay order analysis class diagram

3.4 Unified Analysis Class Diagram

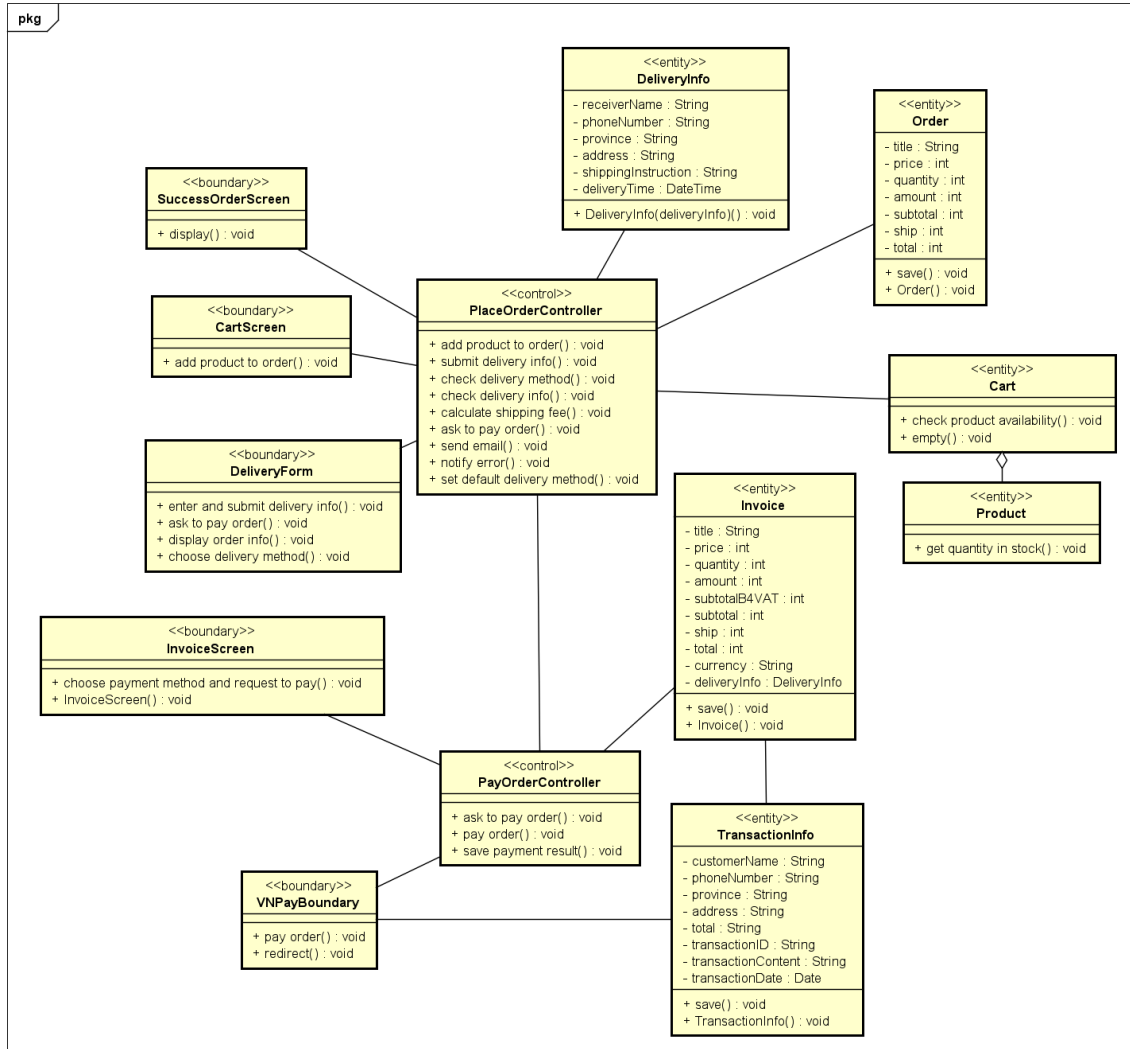


Figure 6 Unified analysis class diagram

3.5 Security Software Architecture

Authentication

The system implements JWT (JSON Web Token) based authentication:

```

interface AuthenticationSystem {
    // Core authentication components
    jwtSecret: string; // Stored securely in environment variables
    tokenExpiration: string = '24h'; // Token validity period
}
  
```

```

// Password handling with bcrypt
saltRounds: number = 10;

async hashPassword(plainPassword: string): Promise<string> {
    return bcrypt.hash(plainPassword, this.saltRounds);
}

async validatePassword(plainPassword: string, hashedPassword: string):
Promise<boolean> {
    return bcrypt.compare(plainPassword, hashedPassword);
}

generateToken(user: User): string {
    return jwt.sign({
        id: user.id,
        roles: user.roles,
        email: user.email
    }, this.jwtSecret, { expiresIn: this.tokenExpiration });
}
}

```

Authentication Flow:

1. User provides credentials (email/password)
2. System validates credentials against bcrypt-hashed passwords in database
3. Upon successful validation, JWT token is generated containing user ID and roles
4. Token is returned to client for subsequent requests
5. Failed login attempts are logged and can trigger account lockout

Authorization

```

interface AuthorizationSystem {
    // Role definitions

```

```

enum UserRole {
  ADMIN = 'ADMIN',
  PRODUCT_MANAGER = 'PRODUCT_MANAGER',
  CUSTOMER = 'CUSTOMER'
}

// Permission mapping
const rolePermissions = {
  ADMIN: [
    'manage_users',
    'reset_passwords',
    'block_users',
    'assign_roles'
  ],
  PRODUCT_MANAGER: [
    'manage_products',
    'manage_orders',
    'view_inventory'
  ],
  CUSTOMER: [
    'view_products',
    'manage_cart',
    'place_orders'
  ]
}

verifyPermission(token: string, requiredPermission: string): boolean {
  const decodedToken = jwt.verify(token, this.jwtSecret);
  return decodedToken.roles.some(role =>

```

```

        rolePermissions[role].includes(requiredPermission)
    );
}
}

```

Authorization Rules:

- Administrators can manage users and assign roles
- Product managers are limited to product and order management
- Multiple roles per user are supported
- Role-based access control enforced at API endpoints
- Specific business constraints (e.g., max 30 product updates per day) implemented at service layer

Encryption

```

interface EncryptionSystem {
    // Data in transit
    const tlsConfig = {
        version: 'TLS 1.3',
        cipherSuites: [
            'TLS_AES_256_GCM_SHA384',
            'TLS_CHACHA20_POLY1305_SHA256'
        ]
    }

    // Sensitive data encryption
    async encryptSensitiveData(data: string): Promise<string> {
        // AES-256-GCM encryption for sensitive data
    }

    async decryptSensitiveData(encryptedData: string): Promise<string> {
        // Corresponding decryption logic
    }
}

```

```
}
```

Encryption Implementation:

- TLS for all API communications
- Sensitive data encrypted at rest:
 - Payment information
 - Personal customer details
 - Authentication credentials
- Strong encryption for payment gateway communication
- Secure key management system

Logging and Auditing

```
interface AuditSystem {
```

```
  // Audit log structure
```

```
  interface AuditLog {
```

```
    timestamp: Date;
```

```
    userId: string;
```

```
    action: string;
```

```
    resource: string;
```

```
    status: string;
```

```
    ipAddress: string;
```

```
    details: object;
```

```
  }
```

```
  async logAction(log: AuditLog): Promise<void> {
```

```
    // Log to secure audit storage
```

```
  }
```

```
  async queryAuditLogs(criteria: object): Promise<AuditLog[]> {
```

```
    // Retrieve filtered audit logs
```

```
  }
```

```
}
```

Logging Requirements:

1. Security Events:

- All authentication attempts
- Role/permission changes
- Password resets
- User account modifications

2. Business Operations:

- Product modifications
- Order status changes
- Payment transactions
- Rush delivery requests

3. System Events:

- API errors
- Performance issues
- System recoveries

4. Audit Trail:

- Tamper-evident logging
- Retention policy compliance
- Secure storage of logs
- Access control for audit data

This security architecture ensures:

- Secure user authentication using industry-standard JWT
- Fine-grained authorization control
- Protection of sensitive data
- Comprehensive audit trail
- Compliance with system requirements
- Scalability for future security enhancements

4 Detailed Design

4.1 User Interface Design

4.1.1 Screen Configuration Standardization

Display

Number of colors supported: 16,777,216 colors

Resolution: 1280 x 832 pixels

Screen

Location of standard buttons: at bottom right corner or bottom left corner or top right corner of the frame

Location of the messages: From the top center down to the bottom of the frame

Screen name position: At the top center of the frame

Consistency in expression of alphanumeric numbers: comma for separator of thousand

Control

Size of the text: small size = 16px, medium size = 20px, large size = 24px, extra large size = 32px

Input check process: Check for emptiness, format

Sequence of moving the focus: Stacking frames

Direct input from keyboard

There will be no shortcuts, and working window can be controlled with the environment utilities(for example, desktop: minimize, collapse, close window; web: close tab, etc.)

Error

Notification of the problem is sent to user whenever errors occur as a popup message or display directly on the screen.

4.1.2 Screen Transition Diagrams

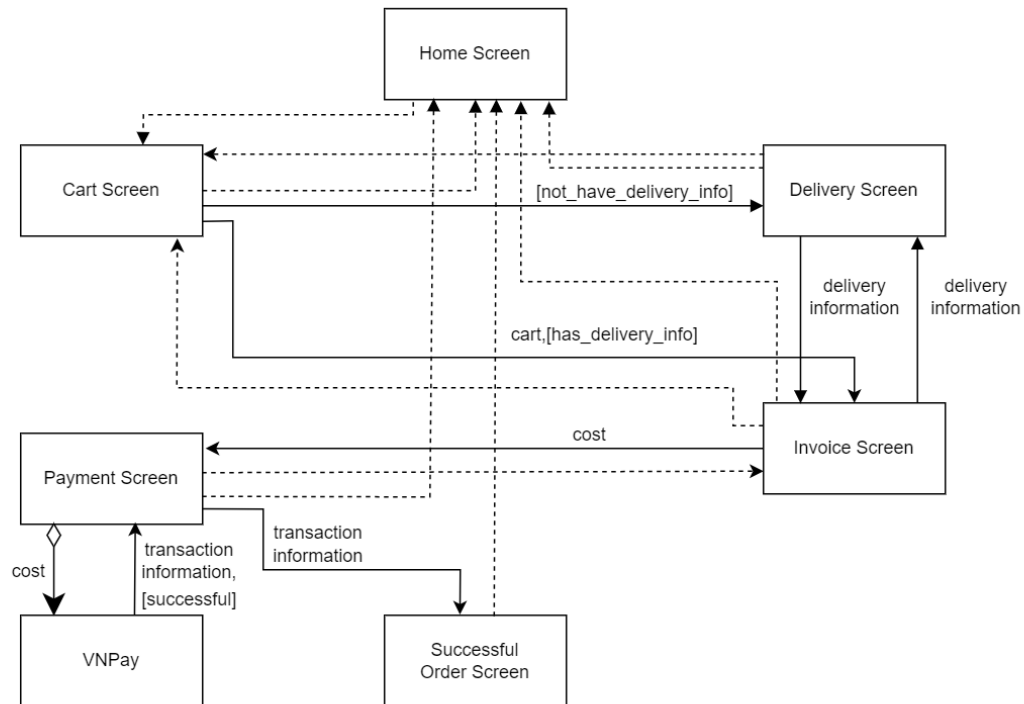


Figure 7 Screen transition diagram

4.1.3 Screen Specifications

Table 1 Home screen specifications

AIMS software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Home Screen	14/10/2024			Nguyễn Trung Chiến
		Control	Operation	Function	
		Area for displaying list of products	Initial	- Display a grid of items either as results of searching or default items.	

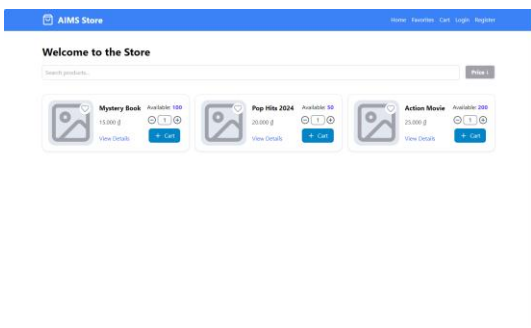
	Header banner	Initial	- Display app logo, search bar, cart button
	Product previews	Initial	- Display preview of item with name, price and quantity in stock
	Searching bar	Input text	- Get text input for searching products
	Add product to cart button	Click	- Add product to cart
	Area for changing the number of products wanted	Input number	- Record the number of products wanted to add to cart
	Search button	Click	- Search products according to description
	Filter button	Click	- Display filter options for searching
	Cart button	Click	- Display cart screen - Display total number of products in cart

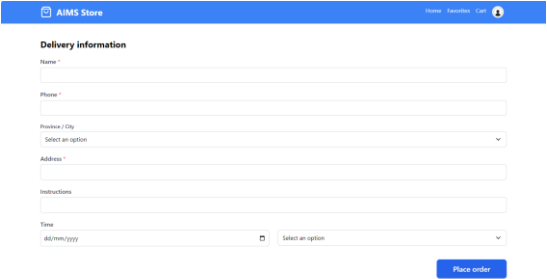
Table 2 Cart screen specifications

AIMS software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Cart Screen	14/10/2024			Nguyễn Trung Chiến

	Control	Operation	Function
	Area for displaying list of products	Initial	- Display a grid of items added to the cart
	Header banner	Initial	- Display app logo, screen name, cart symbol
	Cart symbol	Initial	- Display the total number of products in cart
	Product previews	Initial	- Display preview of item with name, price and quantity
	Text area for displaying subtotal	Initial	- Display the subtotal of products in cart
	Rush checkbox	Click	- Set the type of delivery method wanted for each product. When activated, it indicates rush order option, otherwise, it indicates normal delivery
	Minus button	Click	- Decrease the number of items wanted by 1
	Add button	Click	- Increase the number of items wanted by 1
	Quantity area	Initial	- Display the number of items that

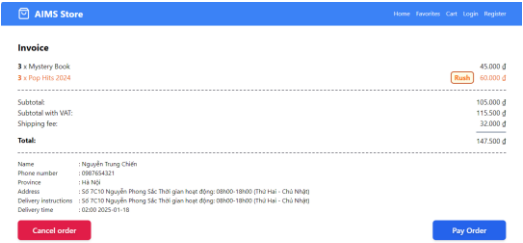
			customer wished to order
	Delete button	Click	- Delete the product from the cart
	Place order button	Click	- Display delivery screen for customer to enter shipping information
	Back button	Click	- Display/go back to home screen

Table 3 Delivery screen specifications

AIMS software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Delivery Screen	14/10/2024			Nguyễn Trung Chiến
		Control	Operation	Function	
		Header banner	Initial	- Display app logo, screen name, home symbol	
		Home symbol	Click	- Display / go back to home screen	
		Area for displaying labels of information fields	Initial	- Display the labels of information fields needed for shipping information	
		Text inputs area for collecting data	Input text	- Collect data corresponding to the labels	

	List choice for province data	Click	- Display a list of provinces and cities for customer to select
	Back button	Click	- Display / go back to cart screen
	Proceed button	Click	- Display invoice screen

Table 4 Invoice screen specifications

AIMS software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Invoice Screen	14/10/2024			Nguyễn Trung Chiến
		Control	Operation	Function	
		Header banner	Initial	- Display app logo, screen name, home symbol	
		Home symbol	Click	- Display / go back to home screen	
		Area for displaying list of products	Initial	- Display list of products with their details (quantity, preview, name, price, delivery method) - If the tag Rush is highlighted, the product (previously chosen to be delivered with rush method) does support rush order,	

			otherwise, the tag is shown in deactivated state (all grey). - If the tag Rush is not highlighted, the product is set to be delivered with normal method
	Area for displaying deliver information	Initial	- Display shipping information
	Edit button in delivery information	Click	- Display / go back to delivery screen so that customer can edit their shipping information
	Area for displaying fees and total cost	Initial	- Display cost of products, related fees and total amount
	Back button	Click	- Display / go back to cart screen
	Pay order	Click	- Display payment screen

Table 5 Payment screen specifications

AIMS software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Payment Screen	14/10/2024			Nguyễn Trung Chiến
		Control	Operation	Function	

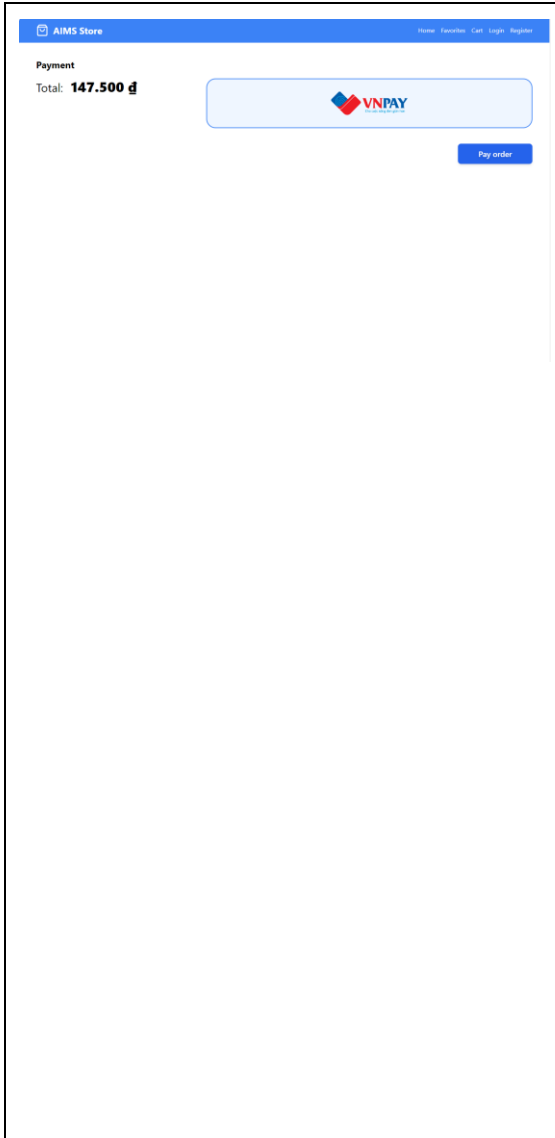
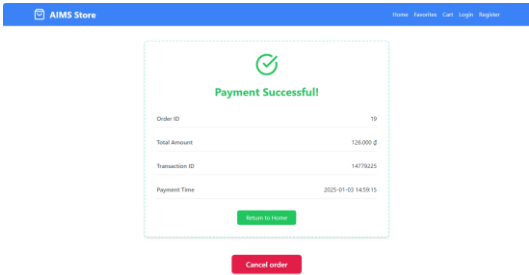
	Header banner	Initial	- Display app logo, screen name, home symbol
	Home symbol	Click	- Display / go back to home screen
	Label of payment method	Initial	- Display the label of payment method
	Area for displaying options for payment	Initial	- Display the method of payment. In this scope, VNPay is chosen and there is no other methods/
	Area for displaying costs	Initial	- Display the costs of the order
	Back button	Click	- Display / go back to invoice screen
	Proceed button	Click	- Forward payment request with needed information to VNPay

Table 6 Successful order screen specifications

AIMS software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Successful Order Screen	14/10/2024			Nguyễn Trung Chiến
		Control	Operation	Function	
		Header banner	Initial	- Display app logo, screen name, home symbol	

	Home symbol	Click	- Display / go back to home screen
	Area for displaying successful notification	Initial	- Display notification of successful order
	Area for displaying general information and transaction information of the order	Initial	- Display general information of the order: customer name, phone number, shipping address, province, total amount; transaction ID, transaction content and transaction datetime
	Home button	Click	- Display / go back home screen

Field attributes

Table 7 Home screen attributes

Screen name	Home screen			
Item name	Number of digits (bytes)	Type	Field attribute	Remarks
Product name	100	Character	Black (bold)	Left justified, preview maximum of 16 characters and reduced with ellipses
Unit price	10	Numeral	Black	Left justified
Currency	5	Character	Black	Next to unit price
Stock quantity	5	Numeral	Black	
Search input	100	Character	Black	Left justified
Cart count	2	Numeral	White (grey border)	After surpassing 99 items in the cart, display +99
Quantity to add	3	Numeral	Black	

Table 8 Cart screen attributes

Screen name	Cart screen			
Item name	Number of digits (bytes)	Type	Field attribute	Remarks
Product name	100	Character	Black (bold)	Left justified, preview maximum of 16 characters and reduced with ellipses
Unit price	10	Numeral	Black	Left justified
Currency	5	Character	Black (bold)	Next to unit price
Cart count	2	Numeral	White (grey border)	After surpassing 99 items in the cart, display +99

Quantity of product	3	Numeral	Black	
Subtotal amount	10	Numeral	Black (bold)	Centered, in the bottom of the screen,

Table 9 Delivery screen attributes

Screen name	Delivery screen			
Item name	Number of digits (bytes)	Type	Field attribute	Remarks
Customer name	30	Character	Black	Left justified
Phone number	10	Character	Black	Left justified
Address	100	Character	Black	Left justified
Shipping instruction	120	Character	Black	Left justified
Time	60	Character	Black	Left justified

Table 10 Invoice screen attributes

Screen name	Invoice screen			
Item name	Number of digits (bytes)	Type	Field attribute	Remarks
Product name	100	Character	Black (bold)	Left justified, preview maximum of 60 characters and reduced with ellipses
Unit price	10	Numeral	Black	Left justified
Currency	5	Character	Black	Next to price
Quantity of product	3	Numeral	Black	
Subtotal amount	10	Numeral	Black (bold)	Right justified

VAT	10	Numeral	Black (bold)	Right justified
Rush delivery fee	10	Numeral	Black (bold)	Right justified
Normal deliver fee	10	Numeral	Black (bold)	Right justified
Total cost	10	Numeral	Black (bold)	Right justified
Customer name	30	Character	Grey	Left justified
Phone number	10	Character	Grey	Left justified
Province	30	Character	Grey	
Address	100	Character	Grey	Left justified
Shipping instruction	120	Character	Grey	Left justified
Time	60	Character	Grey	Left justified

Table 11 Payment screen attributes

Screen name	Payment screen			
Item name	Number of digits (bytes)	Type	Field attribute	Remarks
Subtotal amount	10	Numeral	Black (bold)	Right justified
VAT	10	Numeral	Black (bold)	Right justified
Rush delivery fee	10	Numeral	Black (bold)	Right justified
Normal deliver fee	10	Numeral	Black (bold)	Right justified
Total cost	10	Numeral	Black (bold)	Right justified
Currency	5	Character	Black (bold)	Next to the cost

Table 12 Successful order screen attributes

Screen name	Successful order screen			
Item name	Number of digits (bytes)	Type	Field attribute	Remarks
Customer name	30	Character	Grey	Left justified
Phone number	10	Character	Grey	Left justified
Province	30	Character	Grey	Left justified
Address	100	Character	Grey	Left justified
Transaction ID	12	String	Grey	Left justified
Transaction content	100	Character	Grey	Left justified
Transaction time	60	Character	Grey	Left justified
Total cost	10	Numeral	Black (bold)	Right justified
Currency	5	Character	Black (bold)	Next to total cost

4.2 Data Modeling

4.2.1 Conceptual Data Modeling

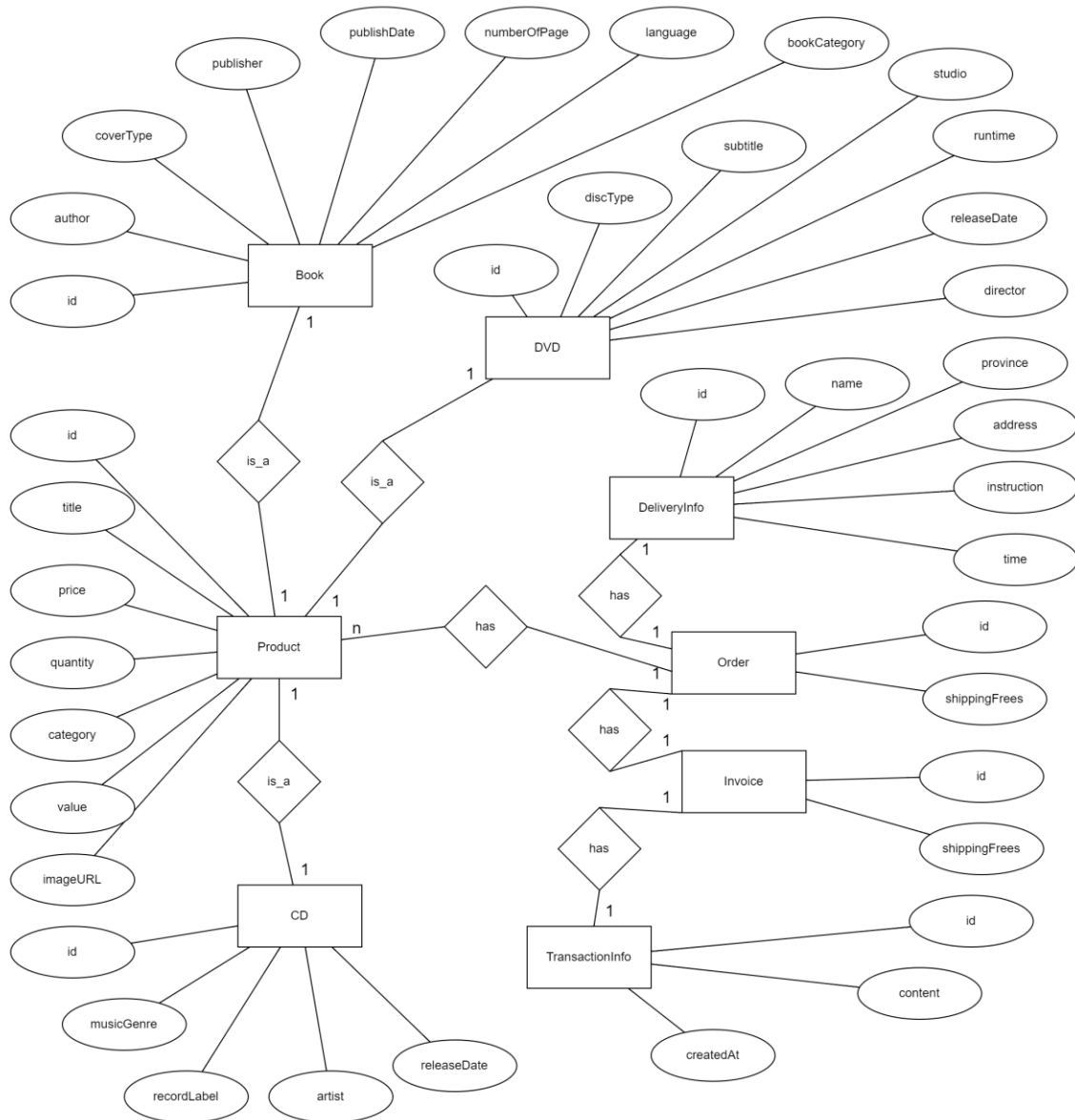


Table 13 Conceptual ERD

4.2.2 Database Design

4.2.2.1 Database Management System

MySQL through XAMPP is an excellent choice for database management because:

1. Integration and Ease of Setup
 - XAMPP provides MySQL (MariaDB) as part of its integrated package

- One-click installation process that automatically configures MySQL with Apache and PHP
- Comes pre-configured with phpMyAdmin for visual database management

2. Key Features of MySQL in XAMPP

- Open-source relational database management system
- Supports complex queries and joins
- Built-in security features with user authentication
- ACID (Atomicity, Consistency, Isolation, Durability) compliant
- Supports multiple storage engines (InnoDB, MyISAM)
- Cross-platform compatibility (Windows, Linux, macOS)

3. Technical Benefits

- Efficient handling of large datasets
- Support for transactions and foreign keys
- Built-in backup and recovery tools
- Multiple user access with privilege control
- Support for various programming languages (PHP, Python, Java)

4. Development Advantages

- Perfect for local development environment
- Easy transition from development to production
- Active community support and extensive documentation
- Regular updates and security patches
- Compatibility with most web hosting services

4.2.2.2 Database Diagram

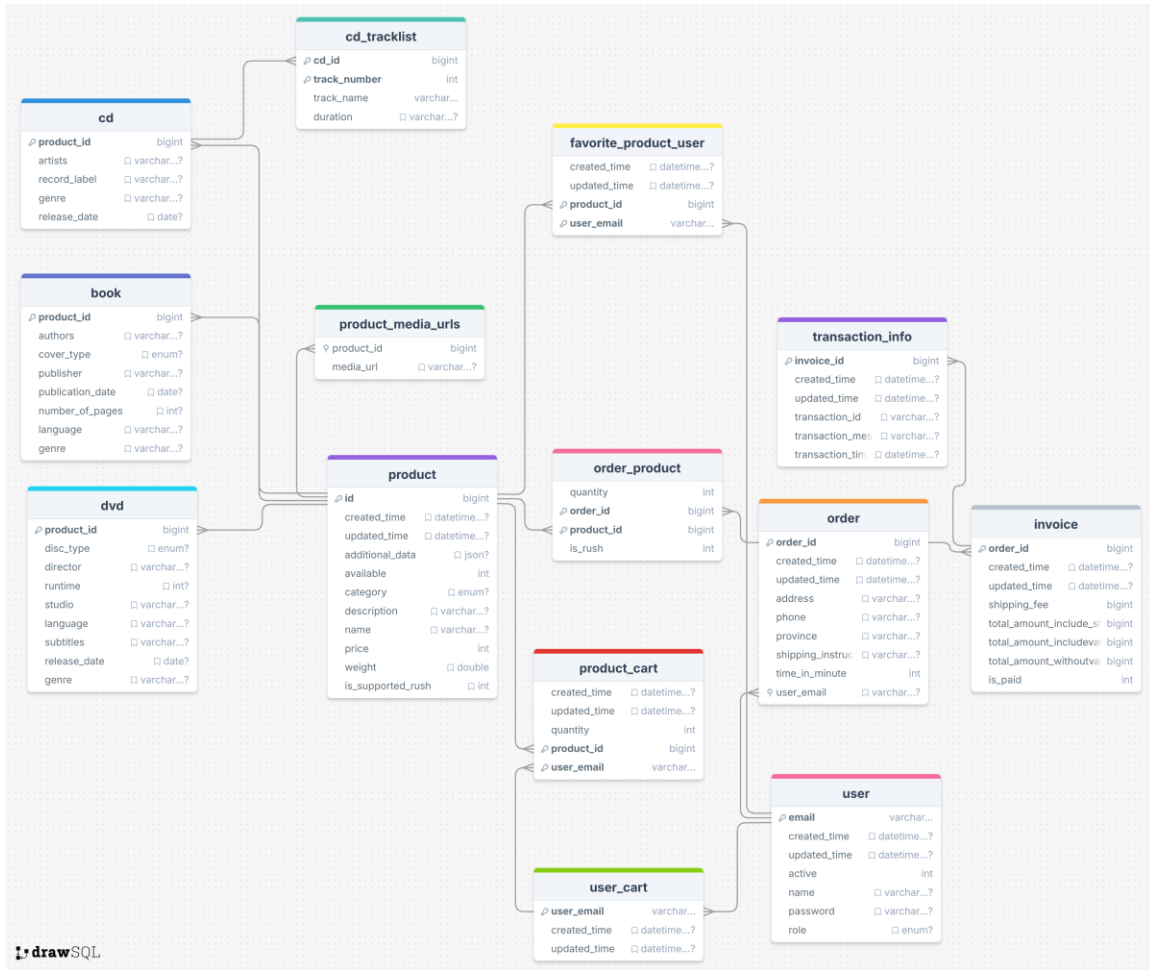


Figure 8 Database design

4.2.2.3 Database Detail Design

Table 14. User table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1	x		email	VARCHAR(255)	NULL	Yes	User's email address, serves as primary identifier
2			created_time	DATETIME(6)	NULL	No	Timestamp when the user record was created
3			updated_time	DATETIME(6)	NULL	No	Timestamp when the user record was last updated

4			active	BIT(1)	NULL	Yes	Flag indicating if the user account is active
5			name	VARCHAR(255)	NULL	No	User's full name
6			password	VARCHAR(255)	NULL	No	User's hashed password
7			role	ENUM	NULL	No	User role: ADMIN, CUSTOMER, or PRODUCT_MANAGER

Table 15 Product table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1	x		id	BIGINT	AUTO_INCREMENT	Yes	Unique identifier for the product
2			created_time	DATETIME(6)	NULL	No	Timestamp when the product was created
3			updated_time	DATETIME(6)	NULL	No	Timestamp when the product was last updated
4			additional_data	JSON	NULL	No	Additional product information in JSON format
5			available	INT	NULL	Yes	Available quantity in stock
6			category	ENUM	NULL	No	Product category: BOOK, CD, or DVD
7			description	VARCHAR(255)	NULL	No	Product description
8			name	VARCHAR(255)	NULL	No	Product name
9			price	INT	NULL	Yes	Product price
10			weight	DOUBLE	0.0	Yes	Product weight
11			is_supported_rush	BIT(1)	1	Yes	Flag indicating if rush delivery is supported

Table 16 Product_media_urls table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1		x	product_id	BIGINT	NULL	Yes	Reference to product table
2			media_url	VARCHAR(255)	NULL	No	URL to product media

Table 17 Order table

#	P K	F K	Column name	Data type	Default value	Mandatory	Description
1	x		order_id	BIGINT	AUTO_INCREMENT	Yes	Unique identifier for the order
2			created_time	DATETIME(6)	NULL	No	Timestamp when order was created
3			updated_time	DATETIME(6)	NULL	No	Timestamp when order was last updated
4			address	VARCHAR(255)	NULL	No	Delivery address
5			phone	VARCHAR(255)	NULL	No	Contact phone number
6			province	VARCHAR(255)	NULL	No	Delivery province
7			shipping_instruction	VARCHAR(255)	NULL	No	Special delivery instructions
8			time_in_minute	INT	NULL	Yes	Delivery time in minutes
9		x	user_email	VARCHAR(255)	NULL	No	Reference to user table

Table 18 Product_cart table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
---	----	----	-------------	-----------	---------------	-----------	-------------

1	x	x	product_id	BIGINT	NULL	Yes	Reference to product table
2	x	x	user_email	VARCHAR(255)	NULL	Yes	Reference to user_cart table
3			created_time	DATETIME(6)	NULL	No	Timestamp when cart item was created
4			updated_time	DATETIME(6)	NULL	No	Timestamp when cart item was updated
5			quantity	INT	NULL	Yes	Quantity of product in cart

Table 19 Invoice table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1	x	x	order_id	BIGINT	NULL	Yes	Reference to order table
2			created_time	DATETIME(6)	NULL	No	Timestamp when invoice was created
3			updated_time	DATETIME(6)	NULL	No	Timestamp when invoice was updated
4			shipping_fee	BIGINT	NULL	Yes	Delivery fee amount
5			total_amount_include_shipping_fee	BIGINT	NULL	Yes	Total amount including shipping
6			total_amount_includevat	BIGINT	NULL	Yes	Total amount including VAT
7			total_amount_withoutvat	BIGINT	NULL	Yes	Total amount excluding VAT
8			is_paid	BIT(1)	0	Yes	Flag indicating if invoice is paid

Table 20 Order_product table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1	x	x	order_id	BIGINT	NULL	Yes	Reference to order table

2	x	x	product_id	BIGINT	NULL	Yes	Reference to product table
3			quantity	INT	NULL	Yes	Quantity of product ordered
4			is_rush	BIT(1)	NULL	Yes	Flag indicating if rush delivery

Table 21 Transaction_info table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1	x	x	invoice_id	BIGINT	NULL	Yes	Reference to invoice table
2			created_time	DATETIME(6)	NULL	No	Timestamp when transaction was created
3			updated_time	DATETIME(6)	NULL	No	Timestamp when transaction was updated
4			transaction_id	VARCHAR(255)	NULL	No	External transaction identifier
5			transaction_message	VARCHAR(255)	NULL	No	Transaction status message
6			transaction_time	DATETIME(6)	NULL	No	Time when transaction occurred

Table 22 Favorite_product_user table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1	x	x	product_id	BIGINT	NULL	Yes	Reference to product table
2	x	x	user_email	VARCHAR(255)	NULL	Yes	Reference to user table
3			created_time	DATETIME(6)	NULL	No	Timestamp when favorite was created

4			updated_time	DATETIME(6)	NULL	No	Timestamp when favorite was updated
---	--	--	--------------	-------------	------	----	-------------------------------------

Table 23 Book table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1	x	x	product_id	BIGINT	NULL	Yes	Reference to product table, serves as primary key
2			authors	VARCHAR(255)	NULL	No	Author(s) of the book
3			cover_type	ENUM	NULL	No	Type of book cover: PAPERBACK or HARDCOVER
4			publisher	VARCHAR(255)	NULL	No	Publisher of the book
5			publication_date	DATE	NULL	No	Date when book was published
6			number_of_pages	INT	NULL	No	Total number of pages in the book
7			language	VARCHAR(50)	NULL	No	Language the book is written in
8			genre	VARCHAR(100)	NULL	No	Genre or category of the book

Table 24 CD table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1	x	x	product_id	BIGINT	NULL	Yes	Reference to product table, serves as primary key
2			artists	VARCHAR(255)	NULL	No	Artist(s) performing on the CD
3			record_label	VARCHAR(255)	NULL	No	Record label that published the CD
4			genre	VARCHAR(100)	NULL	No	Musical genre of the CD
5			release_date	DATE	NULL	No	Date when CD was released

Table 25 CD_tracklist table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1	x	x	cd_id	BIGINT	NULL	Yes	Reference to cd table
2	x		track_number	INT	NULL	Yes	Position of track in album sequence
3			track_name	VARCHAR(255)	NULL	Yes	Name of the track
4			duration	VARCHAR(10)	NULL	No	Length of the track (e.g., "4:32")

Table 26 DVD table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1	x	x	product_id	BIGINT	NULL	Yes	Reference to product table, serves as primary key
2			disc_type	ENUM	NULL	No	Type of DVD: BLURAY or HD_DVD
3			director	VARCHAR(255)	NULL	No	Director(s) of the content
4			runtime	INT	NULL	No	Total runtime in minutes
5			studio	VARCHAR(255)	NULL	No	Studio that produced the content
6			language	VARCHAR(50)	NULL	No	Primary language of the content
7			subtitles	VARCHAR(255)	NULL	No	Available subtitle languages
8			release_date	DATE	NULL	No	Date when DVD was released
9			genre	VARCHAR(100)	NULL	No	Genre or category of the content

Table 27 Review table

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1	x		id	BIGINT	AUTO_INCREMENT	Yes	Unique identifier for review
2			created_time	DATETIME(6)	NULL	No	Timestamp when review was created
3			updated_time	DATETIME(6)	NULL	No	Timestamp when review was updated
4			content	VARCHAR(255)	NULL	No	Review content
5			star	INT	NULL	Yes	Rating in stars
6		x	product_id	BIGINT	NULL	No	Reference to product table
7		x	user_email	VARCHAR(255)	NULL	No	Reference to user table

Table 28 Review_media_urls

#	PK	FK	Column name	Data type	Default value	Mandatory	Description
1		x	review_id	BIGINT	NULL	Yes	Reference to review table
2			media_url	VARCHAR(255)	NULL	No	URL to review media

Script

```

CREATE TABLE IF NOT EXISTS `user` (
    `email` VARCHAR(255) COLLATE utf8mb4_general_ci NOT NULL,
    `created_time` DATETIME(6) DEFAULT NULL,
    `updated_time` DATETIME(6) DEFAULT NULL,
    `active` BIT(1) NOT NULL,
    `name` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
    `password` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
    `role` ENUM('ADMIN', 'CUSTOMER', 'PRODUCT_MANAGER') COLLATE utf8mb4_general_ci DEFAULT NULL,
    PRIMARY KEY(`email`)

```

```
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =  
utf8mb4_general_ci;
```

```
CREATE TABLE IF NOT EXISTS `product` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT,  
  `created_time` DATETIME(6) DEFAULT NULL,  
  `updated_time` DATETIME(6) DEFAULT NULL,  
  `additional_data` JSON DEFAULT NULL,  
  `available` INT NOT NULL,  
  `category` ENUM('BOOK', 'CD', 'DVD') COLLATE utf8mb4_general_ci DEFAULT  
NULL,  
  `description` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,  
  `name` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,  
  `price` INT NOT NULL,  
  `weight` DOUBLE NOT NULL DEFAULT 0.0,  
  `is_supported_rush` BIT(1) NOT NULL DEFAULT 1,  
  PRIMARY KEY(`id`)  
) ENGINE = InnoDB AUTO_INCREMENT = 4 DEFAULT CHARSET = utf8mb4  
COLLATE = utf8mb4_general_ci;
```

-- Book table inheriting from product

```
CREATE TABLE IF NOT EXISTS `book` (  
  `product_id` BIGINT NOT NULL,  
  `authors` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,  
  `cover_type` ENUM('PAPERBACK', 'HARDCOVER') COLLATE  
utf8mb4_general_ci DEFAULT NULL,  
  `publisher` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,  
  `publication_date` DATE DEFAULT NULL,  
  `number_of_pages` INT DEFAULT NULL,  
  `language` VARCHAR(50) COLLATE utf8mb4_general_ci DEFAULT NULL,
```

```

`genre` VARCHAR(100) COLLATE utf8mb4_general_ci DEFAULT NULL,
PRIMARY KEY (`product_id`),
CONSTRAINT `FK_book_product`
    FOREIGN KEY (`product_id`)
    REFERENCES `product` (`id`)
    ON DELETE CASCADE
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =
utf8mb4_general_ci;

```

-- CD table inheriting from product

```

CREATE TABLE IF NOT EXISTS `cd` (
    `product_id` BIGINT NOT NULL,
    `artists` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
    `record_label` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
    `genre` VARCHAR(100) COLLATE utf8mb4_general_ci DEFAULT NULL,
    `release_date` DATE DEFAULT NULL,
    PRIMARY KEY (`product_id`),
    CONSTRAINT `FK_cd_product`
        FOREIGN KEY (`product_id`)
        REFERENCES `product` (`id`)
        ON DELETE CASCADE
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =
utf8mb4_general_ci;

```

-- CD tracklist table (one-to-many relationship with CD)

```

CREATE TABLE IF NOT EXISTS `cd_tracklist` (
    `cd_id` BIGINT NOT NULL,
    `track_number` INT NOT NULL,
    `track_name` VARCHAR(255) COLLATE utf8mb4_general_ci NOT NULL,

```



```

`duration` VARCHAR(10) COLLATE utf8mb4_general_ci DEFAULT NULL,
PRIMARY KEY (`cd_id`, `track_number`),
CONSTRAINT `FK_tracklist_cd`
    FOREIGN KEY (`cd_id`)
    REFERENCES `cd` (`product_id`)
    ON DELETE CASCADE
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =
utf8mb4_general_ci;

-- DVD table inheriting from product
CREATE TABLE IF NOT EXISTS `dvd` (
    `product_id` BIGINT NOT NULL,
    `disc_type` ENUM('BLURAY', 'HD_DVD') COLLATE utf8mb4_general_ci
    DEFAULT NULL,
    `director` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
    `runtime` INT DEFAULT NULL, -- in minutes
    `studio` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
    `language` VARCHAR(50) COLLATE utf8mb4_general_ci DEFAULT NULL,
    `subtitles` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
    `release_date` DATE DEFAULT NULL,
    `genre` VARCHAR(100) COLLATE utf8mb4_general_ci DEFAULT NULL,
    PRIMARY KEY (`product_id`),
    CONSTRAINT `FK_dvd_product`
        FOREIGN KEY (`product_id`)
        REFERENCES `product` (`id`)
        ON DELETE CASCADE
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =
utf8mb4_general_ci;

```

```

CREATE TABLE IF NOT EXISTS `user_cart` (
  `user_email` VARCHAR(255) COLLATE utf8mb4_general_ci NOT NULL,
  `created_time` DATETIME(6) DEFAULT NULL,
  `updated_time` DATETIME(6) DEFAULT NULL,
  PRIMARY KEY(`user_email`),
  CONSTRAINT `FKn8gfn4fx5uag77lvx22j40fap` FOREIGN KEY(`user_email`)
REFERENCES `user`(`email`)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =
utf8mb4_general_ci;

```

```

CREATE TABLE IF NOT EXISTS `product_media_urls` (
  `product_id` BIGINT NOT NULL,
  `media_url` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
  KEY `FK23mquixq35vce83glx1ofyomw`(`product_id`),
  CONSTRAINT `FK23mquixq35vce83glx1ofyomw` FOREIGN KEY(`product_id`)
REFERENCES `product`(`id`)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =
utf8mb4_general_ci;

```

```

CREATE TABLE IF NOT EXISTS `favorite_product_user` (
  `created_time` DATETIME(6) DEFAULT NULL,
  `updated_time` DATETIME(6) DEFAULT NULL,
  `product_id` BIGINT NOT NULL,
  `user_email` VARCHAR(255) COLLATE utf8mb4_general_ci NOT NULL,
  PRIMARY KEY(`product_id`, `user_email`),
  KEY `FK4g5ro7l61ny5mgta47yyua6m9`(`user_email`),
  CONSTRAINT `FK4g5ro7l61ny5mgta47yyua6m9` FOREIGN KEY(`user_email`)
REFERENCES `user`(`email`),
  CONSTRAINT `FK5naqqahk6kxmm3cjynn24ounm` FOREIGN KEY(`product_id`)
REFERENCES `product`(`id`)

```

```
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =  
utf8mb4_general_ci;
```

```
CREATE TABLE IF NOT EXISTS `order` (  
  `order_id` BIGINT NOT NULL AUTO_INCREMENT,  
  `created_time` DATETIME(6) DEFAULT NULL,  
  `updated_time` DATETIME(6) DEFAULT NULL,  
  `address` VARCHAR(255) CHARACTER SET utf8mb4 COLLATE  
utf8mb4_general_ci DEFAULT NULL,  
  `phone` VARCHAR(255) CHARACTER SET utf8mb4 COLLATE  
utf8mb4_general_ci DEFAULT NULL,  
  `province` VARCHAR(255) CHARACTER SET utf8mb4 COLLATE  
utf8mb4_general_ci DEFAULT NULL,  
  `shipping_instruction` VARCHAR(255) CHARACTER SET utf8mb4 COLLATE  
utf8mb4_general_ci DEFAULT NULL,  
  `time_in_minute` INT NOT NULL,  
  `user_email` VARCHAR(255) CHARACTER SET utf8mb4 COLLATE  
utf8mb4_general_ci DEFAULT NULL,  
  PRIMARY KEY (`order_id`),  
  KEY `FKcpl0mjoeqhxvgeeeq5piwpd3i` (`user_email`) USING BTREE,  
  CONSTRAINT `FKcpl0mjoeqhxvgeeeq5piwpd3i`  
    FOREIGN KEY (`user_email`) REFERENCES `user` (`email`)  
)  
ENGINE=InnoDB  
AUTO_INCREMENT=12  
DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;
```

```
CREATE TABLE IF NOT EXISTS `product_cart` (
```

```

`created_time` DATETIME(6) DEFAULT NULL,
`updated_time` DATETIME(6) DEFAULT NULL,
`quantity` INT NOT NULL,
`product_id` BIGINT NOT NULL,
`user_email` VARCHAR(255) COLLATE utf8mb4_general_ci NOT NULL,
PRIMARY KEY(`product_id`, `user_email`),
KEY `FKfag18f06h5h8xpnogiuce5fbc`(`user_email`),
CONSTRAINT `FKfag18f06h5h8xpnogiuce5fbc` FOREIGN KEY(`user_email`)
REFERENCES `user_cart`(`user_email`),
CONSTRAINT `FKhpnrxdy3jhuameyod08ilvvw` FOREIGN KEY(`product_id`)
REFERENCES `product`(`id`)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =
utf8mb4_general_ci;

```

```

CREATE TABLE IF NOT EXISTS `invoice` (
  `order_id` BIGINT NOT NULL,
  `created_time` DATETIME(6) DEFAULT NULL,
  `updated_time` DATETIME(6) DEFAULT NULL,
  `shipping_fee` BIGINT NOT NULL,
  `total_amount_include_shipping_fee` BIGINT NOT NULL,
  `total_amount_includevat` BIGINT NOT NULL,
  `total_amount_withoutvat` BIGINT NOT NULL,
  `is_paid` BIT(1) NOT NULL DEFAULT 0,
  PRIMARY KEY(`order_id`),
  CONSTRAINT `FKr27vrfyll0shs80upv1rmctie` FOREIGN KEY(`order_id`)
REFERENCES `order`(`order_id`)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =
utf8mb4_general_ci;

```

```

CREATE TABLE IF NOT EXISTS `order_product` (

```

```

`quantity` INT NOT NULL,
`order_id` BIGINT NOT NULL,
`product_id` BIGINT NOT NULL,
`is_rush` BIT(1) NOT NULL,
PRIMARY KEY(`order_id`, `product_id`),
KEY `FKhnfgqyx3i80qoymrssls3kno`(`product_id`),
CONSTRAINT `FKhnfgqyx3i80qoymrssls3kno` FOREIGN KEY(`product_id`)
REFERENCES `product`(`id`),
CONSTRAINT `FKm6igrp4lwucj1me05axmv885c` FOREIGN KEY(`order_id`)
REFERENCES `order`(`order_id`)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =
utf8mb4_general_ci;

```

```

CREATE TABLE IF NOT EXISTS `review` (
`id` BIGINT NOT NULL AUTO_INCREMENT,
`created_time` DATETIME(6) DEFAULT NULL,
`updated_time` DATETIME(6) DEFAULT NULL,
`content` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
`star` INT NOT NULL,
`product_id` BIGINT DEFAULT NULL,
`user_email` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
PRIMARY KEY(`id`),
KEY `FKiyof1sindb9qqr9o8npj8klt`(`product_id`),
KEY `FK6hqpdfb0b6wduxedrygq6hv32`(`user_email`),
CONSTRAINT `FK6hqpdfb0b6wduxedrygq6hv32` FOREIGN KEY(`user_email`)
REFERENCES `user`(`email`),
CONSTRAINT `FKiyof1sindb9qqr9o8npj8klt` FOREIGN KEY(`product_id`)
REFERENCES `product`(`id`)
) ENGINE = InnoDB AUTO_INCREMENT = 3 DEFAULT CHARSET = utf8mb4
COLLATE = utf8mb4_general_ci;

```

```

CREATE TABLE IF NOT EXISTS `transaction_info` (
  `invoice_id` BIGINT NOT NULL,
  `created_time` DATETIME(6) DEFAULT NULL,
  `updated_time` DATETIME(6) DEFAULT NULL,
  `transaction_id` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
  `transaction_message` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT
  NULL,
  `transaction_time` DATETIME(6) DEFAULT NULL,
  PRIMARY KEY(`invoice_id`),
  CONSTRAINT `FKq5i7p9l0qnjjfms1iy1d16aq` FOREIGN KEY(`invoice_id`)
  REFERENCES `invoice`(`order_id`)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =
  utf8mb4_general_ci;

```

```

CREATE TABLE IF NOT EXISTS `review_media_urls` (
  `review_id` BIGINT NOT NULL,
  `media_url` VARCHAR(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
  KEY `FKpsmh0fah3dfl3seej7d8hpkbt`(`review_id`),
  CONSTRAINT `FKpsmh0fah3dfl3seej7d8hpkbt` FOREIGN KEY(`review_id`)
  REFERENCES `review`(`id`)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE =
  utf8mb4_general_ci;

```

```

INSERT INTO `product`
  (`id`, `created_time`, `updated_time`, `additional_data`, `available`, `category`,
  `description`, `name`, `price`)
VALUES
  (1, '2024-12-17 13:39:02.000000', '2024-12-17 13:39:02.000000', '{"size": "M", "color":
  "red"}', 100, 'BOOK', 'A thrilling mystery novel', 'Mystery Book', 15000),

```

(2, '2024-12-17 13:39:02.000000', '2024-12-17 13:39:02.000000', '{"size": "L", "color": "blue"}', 50, 'CD', 'Pop music album', 'Pop Hits 2024', 20000),

(3, '2024-12-17 13:39:02.000000', '2024-12-17 13:39:02.000000', '{"size": "XL", "color": "black"}', 200, 'DVD', 'Action movie DVD', 'Action Movie', 25000);

INSERT INTO `product\_media\_urls`

(`product\_id`, `media\_url`)

VALUES

(1, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau-1-1.jpg.webp'),

(2, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau-5-2-1.jpg.webp'),

(3, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau-9-1.jpg.webp'),

(1, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau-2-1.jpg.webp'),

(1, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau-3-1.jpg.webp'),

(1, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau.jpg.webp'),

(2, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau-6-1.jpg.webp'),

(2, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau-8-1.jpg.webp'),

(2, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau-84-1.jpg.webp'),

(3, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau-10-1.jpg.webp'),

(3, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau-11-1.jpg.webp'),

(3, 'https://hoanghamobile.com/tin-tuc/wp-content/webp-express/webp-images/uploads/2024/05/anh-trai-xau-14-1.jpg.webp');

4.3 Non-Database Management System Files

None

4.4 Class Design

4.4.1 General Class Diagram

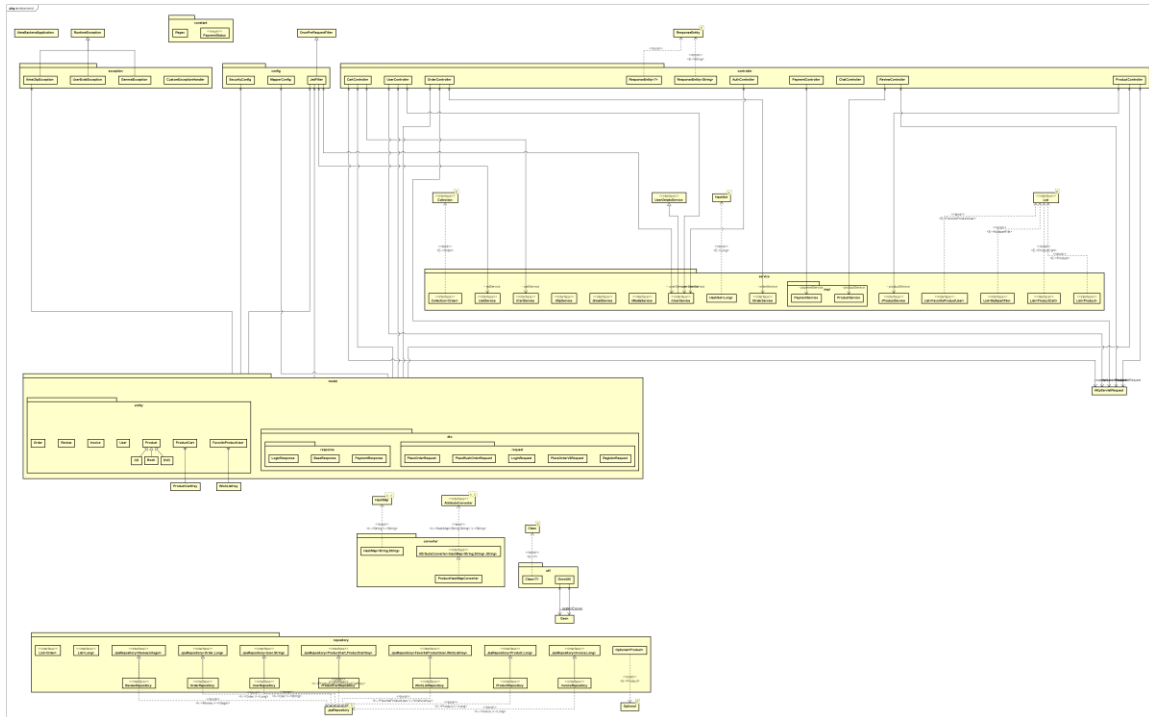
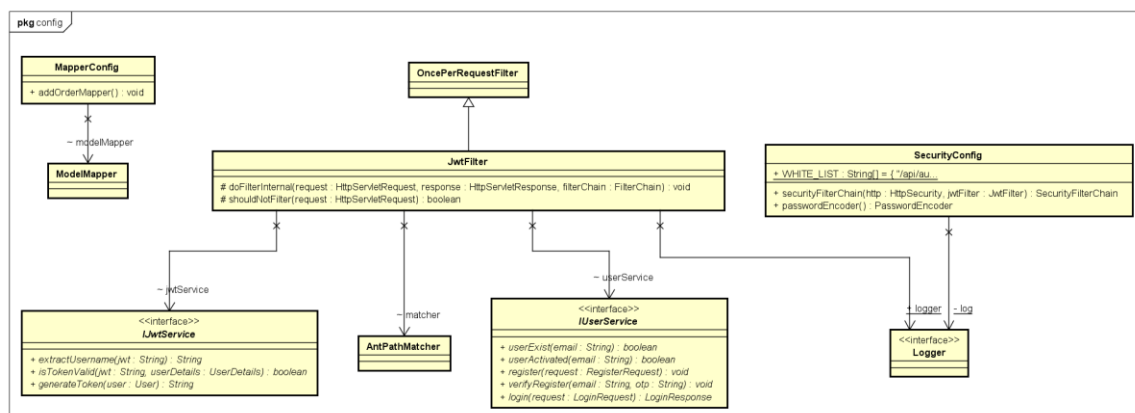


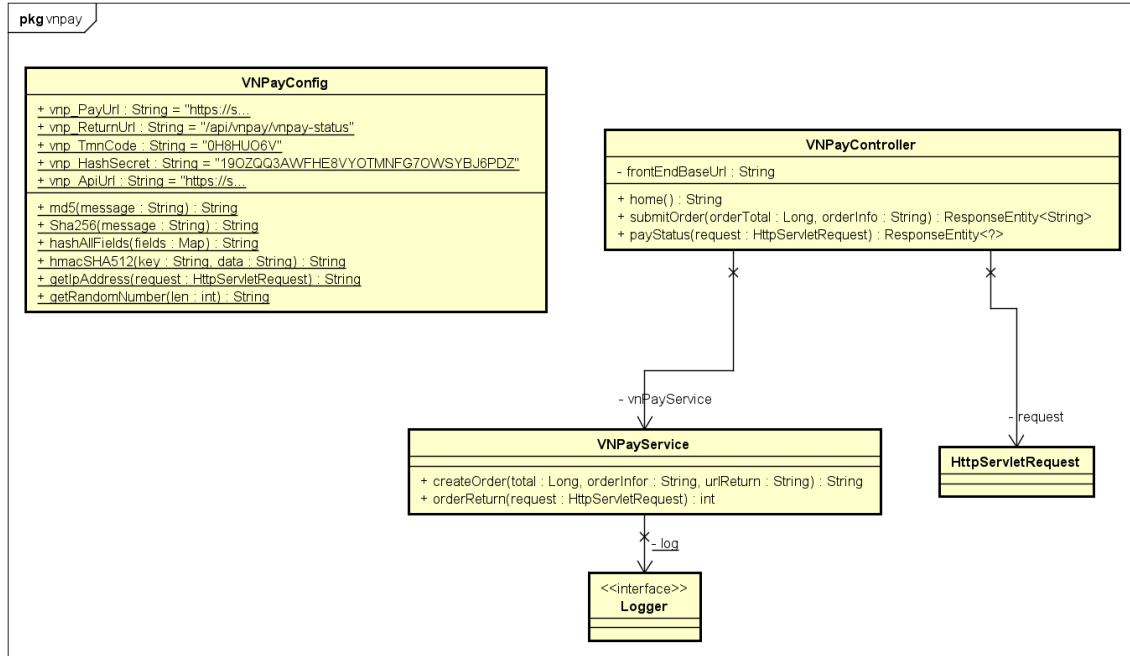
Figure 9 General class diagram

4.4.2 Class Diagrams

4.4.2.1 Class Diagram for Packages



4.4.2.2 Class Diagram for VNPay Subsystem



4.4.3 Class Design

4.4.3.1 AuthController

Table 29 Class design AuthController

Attributes

#	Name	Data type	Default value	Description
1	N/A	N/A	N/A	Handles authentication-related operations such as user registration, verification, and login.

Operations

#	Name	Return type	Description (purpose)
1	register	void	Registers a new user.
2	verifyRegister	void	Verifies the user's registration using OTP.
3	login	LoginResponse	Authenticates the user and returns a JWT token.

4.4.3.2 CartController

Table 30 Class design CartController

Attributes

#	Name	Data type	Default value	Description
1	N/A	N/A	N/A	Manages the user's shopping cart operations.

Operations

#	Name	Return type	Description (purpose)
1	getUserCart	List<ProductCart>	Retrieves the current user's cart.
2	addToCart	void	Adds a product to the user's cart.

4.4.3.3 OrderController

Table 31 Class design OrderController

Attributes

#	Name	Data type	Default value	Description
1	N/A	N/A	N/A	Handles order-related operations such as placing, deleting, and retrieving orders.

Operations

#	Name	Return type	Description (purpose)
1	getOrders	List<Order>	Retrieves the current user's orders.
2	placeOrder	Invoice	Places a new order.
3	placeOrderV2	Invoice	Places a new order with version 2 request.
4	placeRushOrder	Invoice	Places a rush order.
5	deleteOrder	void	Deletes an order.
6	payOrder	void	Placeholder for order payment.

4.4.3.4 PaymentController

Table 32 Class design PaymentController

Attributes

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	N/A	N/A	N/A	Manages payment-related operations.
---	-----	-----	-----	-------------------------------------

Operations

#	Name	Return type	Description (purpose)
1	createPayment	String	Creates a payment URL.
2	paymentStatus	PaymentResponse	Processes the payment return status.

4.4.3.5 ProductController

Table 33 Class design ProductController

Attributes

#	Name	Data type	Default value	Description
1	N/A	N/A	N/A	Handles product-related operations such as retrieving, searching, and managing wish lists.

Operations

#	Name	Return type	Description (purpose)
1	getProduct	Product	Retrieves a product by its ID.
2	getAllProduct	List<Product>	Retrieves all products.
3	searchProduct	List<Product>	Placeholder for product search.
4	getWishList	List<FavoriteProductUser>	Retrieves the user's wish list.
5	addWishList	void	Adds a product to the user's wish list.

4.4.3.6 UserController

Table 34 Class design UserController

Attributes

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	N/A	N/A	N/A	Handles user-related operations.
---	-----	-----	-----	----------------------------------

Operations

#	Name	Return type	Description (purpose)
1	getInfo	UserDto	Retrieves the current user's information.

4.4.3.7 BaseEntity

Table 35 Class design BaseEntity

Attributes

#	Name	Data type	Default value	Description
1	createdTime	LocalDateTime	N/A	The time when the entity was created.
2	updatedAt	LocalDateTime	N/A	The time when the entity was last updated.

Operations

#	Name	Return type	Description (purpose)
1	N/A	N/A	N/A

4.4.3.8 Book

Table 36 Class design Book

Attributes

#	Name	Data type	Default value	Description
1	authors	List<String>	N/A	The authors of the book.
2	coverType	String	N/A	The type of cover of the book.
3	publisher	String	N/A	The publisher of the book.
4	publicationDate	LocalDate	N/A	The publication date of the book.
5	numberOfPages	int	N/A	The number of pages in the book.

6	language	String	N/A	The language of the book.
7	genre	String	N/A	The genre of the book.

4.4.3.9 CD

Table 37 Class design CD

Attributes

#	Name	Data type	Default value	Description
1	artists	List<String>	N/A	The artists of the CD.
2	recordLabel	String	N/A	The record label of the CD.
3	genre	String	N/A	The genre of the CD.
4	releaseDate	LocalDate	N/A	The release date of the CD.
5	tracklist	List<String>	N/A	The tracklist of the CD.

4.4.3.10 DVD

Table 38 Class design DVD

Attributes

#	Name	Data type	Default value	Description
1	discType	String	N/A	The type of disc of the DVD.
2	director	String	N/A	The director of the DVD.
3	runtime	int	N/A	The runtime of the DVD.
4	studio	String	N/A	The studio of the DVD.
5	language	String	N/A	The language of the DVD.

6	subtitles	List<String>	N/A	The subtitles available on the DVD.
7	releaseDate	LocalDate	N/A	The release date of the DVD.
8	genre	String	N/A	The genre of the DVD.

4.4.3.11 FavoriteProductUser

Table 39 Class design FavoriteProductUser

Attributes

#	Name	Data type	Default value	Description
1	key	Long	N/A	The key of the favorite product user.
2	user	User	N/A	The user who favorited the product.
3	product	Product	N/A	The product that was favorited.

4.4.3.12 Invoice

Table 40 Class design Invoice

Attributes

#	Name	Data type	Default value	Description
1	orderId	Long	N/A	The ID of the order associated with the invoice.
2	order	Order	N/A	The order associated with the invoice.
3	isPaid	boolean	N/A	Whether the invoice has been paid.
4	shippingFee	BigDecimal	N/A	The shipping fee for the order.
5	totalAmountWithoutVAT	BigDecimal	N/A	The total amount without VAT.

6	totalAmountIncludeVAT	BigDecimal	N/A	The total amount including VAT.
7	totalAmountIncludeShippingFee	BigDecimal	N/A	The total amount including the shipping fee.

4.4.3.13 Order

Table 41 Class design Order

Attributes

#	Name	Data type	Default value	Description
1	orderId	Long	N/A	The ID of the order.
2	user	User	N/A	The user who placed the order.
3	orderProducts	List<OrderProduct>	N/A	The products in the order.
4	address	String	N/A	The shipping address for the order.
5	phone	String	N/A	The phone number for the order.
6	province	String	N/A	The province for the order.
7	shippingInstruction	String	N/A	The shipping instructions for the order.
8	isRush	boolean	N/A	Whether the order is a rush order.
9	timeInMinute	int	N/A	The time in minutes for the rush order.

4.4.3.14 *OrderProduct*

Table 42 Class design OrderProduct

Attributes

#	Name	Data type	Default value	Description
1	key	Long	N/A	The key of the order product.
2	product	Product	N/A	The product in the order.
3	order	Order	N/A	The order associated with the product.
4	quantity	int	N/A	The quantity of the product in the order.
5	isRush	boolean	N/A	Whether the product is part of a rush order.

4.4.3.15 *Product*

Table 43 Class design Product

Attributes

#	Name	Data type	Default value	Description
1	id	Long	N/A	The ID of the product.
2	name	String	N/A	The name of the product.
3	description	String	N/A	The description of the product.
4	available	boolean	N/A	Whether the product is available.
5	price	BigDecimal	N/A	The price of the product.
6	weight	BigDecimal	N/A	The weight of the product.
7	additionalData	Map<String, String>	N/A	Additional data for the product.

8	category	String	N/A	The category of the product.
9	isSupportedRush	boolean	N/A	Whether the product supports rush delivery.
10	mediaUrls	List<String>	N/A	The media URLs for the product.

4.4.3.16 *ProductCart*

Table 44 Class design ProductCart

Attributes

#	Name	Data type	Default value	Description
1	key	Long	N/A	The key of the product cart.
2	quantity	int	N/A	The quantity of the product in the cart.
3	product	Product	N/A	The product in the cart.
4	user	User	N/A	The user who owns the cart.

4.4.3.17 *TransactionInfo*

Table 45 Class design TransactionInfo

Attributes

#	Name	Data type	Default value	Description
1	transactionId	Long	N/A	The ID of the transaction.
2	transactionTime	LocalDateTime	N/A	The time of the transaction.
3	transactionMessage	String	N/A	The message of the transaction.
4	invoiceId	Long	N/A	The ID of the invoice associated with the transaction.
5	invoice	Invoice	N/A	The invoice associated with the transaction.

4.4.3.18 User

Table 46 Class design User

Attributes

#	Name	Data type	Default value	Description
1	email	String	N/A	The email of the user.
2	name	String	N/A	The name of the user.
3	password	String	N/A	The password of the user.
4	role	String	N/A	The role of the user.
5	active	boolean	N/A	Whether the user is active.

4.4.3.19 CartService

Table 47 Class design CartService

Operations

#	Name	Return type	Description (purpose)
1	getUserCart	List<ProductCart>	Retrieves the user's cart.
2	addToCart	void	Adds a product to the user's cart.

4.4.3.20 EmailService

Table 48 Class design EmailService

Operations

#	Name	Return type	Description (purpose)
1	sendRegisterEmail	void	Sends a registration email.
2	sendOrderEmail	void	Sends an order email.
3	sendMail	void	Sends an email.

4.4.3.21 *JwtService*

Table 49 Class design *JwtService*

Operations

#	Name	Return type	Description (purpose)
1	extractUsername	String	Extracts the username from a JWT token.
2	generateToken	String	Generates a JWT token.
3	isTokenValid	boolean	Validates a JWT token.

4.4.3.22 *MediaService*

Table 50 Class design *MediaService*

Operations

#	Name	Return type	Description (purpose)
1	saveMedia	String	Saves a media file.

4.4.3.23 *EmailService*

Table 51 Class design *EmailService*

#	Name	Data type	Default value	Description
1	mailSender	JavaMailSender	N/A	Mail sender for sending emails.

#	Name	Return type	Description (purpose)
1	sendRegisterEmail	void	Sends a registration email.
2	sendOrderEmail	void	Sends an order email.
3	sendMail	void	Sends an email.

4.4.3.24 UserService

Table 52 Class design UserService

Attributes

#	Name	Data type	Default value	Description
1	userRepository	UserRepository	N/A	Repository for accessing user data.
2	otpService	IOtpService	N/A	Service for handling OTP operations.
3	emailService	IEmailService	N/A	Service for handling email operations.
4	jwtService	IJwtService	N/A	Service for handling JWT operations.
5	HttpServletRequest	HttpServletRequest	N/A	HTTP servlet request.
6	passwordEncoder	PasswordEncoder	N/A	Encoder for user passwords.
7	productCartRepository	ProductCartRepository	N/A	Repository for accessing product cart data.

Operations

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

1	userExist	boolean	Checks if a user exists by email.
2	userActivated	boolean	Checks if a user is activated by email.
3	register	void	Registers a new user.
4	verifyRegister	void	Verifies the user's registration using OTP.
5	login	LoginResponse	Authenticates the user and returns a JWT token.
6	loadUserByUsername	UserDetails	Loads a user by username.

4.4.3.25 *ProductService*

Table 53 Class design ProductService

Attributes

#	Name	Data type	Default value	Description
1	reviewRepository	ReviewRepository	N/A	Repository for accessing review data.
2	HttpServletRequest	HttpServletRequest	N/A	HTTP servlet request.
3	productRepository	ProductRepository	N/A	Repository for accessing product data.

4	mediaService	IMediaService	N/A	Service for handling media operations.
5	wishListRepository	WishListRepository	N/A	Repository for accessing wish list data.
6	maxContentReview	int	N/A	Maximum content length for reviews.
7	maxMediaCountReview	int	N/A	Maximum media count for reviews.
8	supportedMediaTypeReview	List	N/A	Supported media types for reviews.

Operations

#	Name	Return type	Description (purpose)
1	review	void	Adds a review for a product.
2	getProduct	Product	Retrieves a product by its ID.
3	getAllProducts	List	Retrieves all products.
4	addWishList	void	Adds a product to the user's wish list.
5	getWishList	List	Retrieves the user's wish list.

4.4.3.26 PaymentService

Table 54 Class design PaymentService

Attributes

#	Name	Data type	Default value	Description
1	paymentStrategies	Map<String, PaymentStrategy>	N/A	Map of payment strategies.

Operations

#	Name	Return type	Description (purpose)
1	createPayment	String	Creates a payment URL.
2	processPaymentReturn	PaymentResponse	Processes the payment return status.

4.4.3.27 *OtpService*

Table 55 Class design OtpService

Attributes

#	Name	Data type	Default value	Description
1	OTP_REGISTER_LENGTH	int	6	Length of the OTP for registration.
2	registerOtpAndCreateTime	ConcurrentHashMap<String, String>	N/A	Map of OTPs and their creation times.
3	REGISTER_OTP_EXP	long	N/A	Expiration time for registration OTPs.

Operations

#	Name	Return type	Description (purpose)
1	generateRegisterOtp	String	Generates a registration OTP.
2	checkRegisterOtp	void	Checks the registration OTP.
3	cleanRegisterOtpMap	void	Cleans expired OTPs from the map.

4.4.3.28 OrderService

Table 56 Class design OrderService

Attributes

#	Name	Data type	Default value	Description
1	orderRepository	OrderRepository	N/A	Repository for accessing order data.
2	cartService	ICartService	N/A	Service for handling cart operations.
3	invoiceRepository	InvoiceRepository	N/A	Repository for accessing invoice data.
4	HttpServletRequest	ServletRequest	N/A	HTTP servlet request.
5	productCartRepository	ProductCartRepository	N/A	Repository for accessing product cart data.

Operations

#	Name	Return type	Description (purpose)
1	placeOrder	Invoice	Places a new order.
2	placeOrderV2	Invoice	Places a new order with version 2 request.
3	placeRushOrder	Invoice	Places a rush order.
4	deleteOrder	void	Deletes an order.
5	getOrder	List	Retrieves the user's orders.
6	createNewOrder	Invoice	Creates a new order.
7	createInvoice	Invoice	Creates an invoice for an order.
8	calculateBaseShippingFee	int	Calculates the base shipping fee.
9	calculateShippingFee	int	Calculates the shipping fee for an order.
10	checkOrderNotPaidExist	void	Checks if there are any unpaid orders for the user.
11	checkPlaceOrderRequestInCart	void	Checks if the place order request exists in the cart.

4.4.3.29 *MediaService*

Table 57 Class design *MediaService*

Attributes

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	savedReviewMedia	String	N/A	Folder for saving review media.
---	------------------	--------	-----	---------------------------------

Operations

#	Name	Return type	Description (purpose)
1	saveMedia	String	Saves a media file and returns its relative path.

4.4.3.30 JwtService

Table 58 Class design JwtService

Attributes

#	Name	Data type	Default value	Description
1	SECRET_KEY	String	"123AAAAAAAAAAAAAAAAAAAA"	Secret key for JWT.
2	HttpServletRequest	HttpServletRequest	N/A	HTTP servlet request.
3	expired	long	N/A	Expiration time for JWT.
4	key	Key	N/A	Key for signing JWT.

Operations

#	Name	Return type	Description (purpose)
1	extractUsername	String	Extracts the username from a JWT token.
2	extractClaim	T	Extracts a claim from a JWT token.
3	generateToken	String	Generates a JWT token.

4	isTokenValid	boolean	Validates a JWT token.
5	isTokenExpired	boolean	Checks if a JWT token is expired.
6	extractExpiration	Date	Extracts the expiration date from a JWT token.
7	extractAllClaims	Claims	Extracts all claims from a JWT token.

5 Design Considerations

5.1 Goals and Guidelines

Performance and Reliability Goals:

- System must handle 1,000 concurrent users without performance degradation
- Maximum response time: 2 seconds normal, 5 seconds peak
- 300 hours continuous operation without failure
- 1-hour maximum recovery time after incidents

Security Guidelines:

Product managers limited to:

- Single product addition/edit at a time
- Maximum 10 product deletions at once
- Maximum 30 product updates/deletions per day
- Price updates limited to twice per day
- Price must be between 30-150% of product value

Data Integrity Guidelines:

- Mandatory product information validation
- Input validation for dates and formats
- Stock quantity validation before order processing
- Transaction history tracking

User Experience Guidelines:

- 20 products per page display limit
- Clear cart after successful payment
- Email notifications for administrative actions and orders
- Detailed error notifications for invalid operations

5.2 Architectural Strategies

Technology Stack Selection:

- Database: MySQL (based on SQL schema)
 - Supports concurrent transactions
 - Handles complex relationships between entities
 - Provides data integrity through foreign key constraints

System Architecture:

- Multi-tier architecture:
 - Presentation Layer: Desktop application interface
 - Business Logic Layer: Product, Order, User management
 - Data Access Layer: MySQL database
 - Integration Layer: VNPay payment gateway integration

Data Management:

- Inheritance Strategy for Products:
 - Base product table with common attributes
 - Specialized tables for Books, CDs, DVDs
 - One-to-one relationships with the base product

Security Architecture:

- Role-based access control (ADMIN, CUSTOMER, PRODUCT_MANAGER)
- Authentication required for administrative functions
- Transaction logging for audit trails

Integration Strategy:

- VNPay Sandbox integration for payments
- Email service integration for notifications
- REST APIs for payment gateway communication

Error Handling:

- Input validation at UI and business logic layers
- Transaction rollback for failed operations
- Detailed error messages for users
- Logging of system errors for monitoring

Concurrency Management:

- Database transaction isolation for concurrent orders
- Stock quantity locking during order processing
- Optimistic locking for product updates

Extension Strategy:

- Support for future payment methods

- Extensible product type system
- Modular design for adding new features

State Management:

- Order state tracking (pending, approved, rejected)
- Cart session management
- Payment transaction tracking

Performance Optimization:

- Database indexing for frequent queries
- Connection pooling for database access
- Pagination for product listings
- Caching strategies for product data

Resource Management:

- Image storage for product media
- Session management for user carts
- Database connection pooling
- Memory management for concurrent users

5.3 Coupling and Cohesion

Overall, after refinements and tweaks, the design reach a low level of coupling and a high level of cohesion among the modules. If there were a specific exception, it would be the status of config or utility classes(Regex, SecurityConfig) – common coupling, which is acceptable since the intrinsic behavior of utility classes are designed to be used globally by other modules.

Table 59 Common coupling improvements

Related modules	Description	Improvement Direction
SecurityConfig	Using a static singleton instance WHITE_LIST, share with JwtFilter	Remove WHITE_LIST from JwtFilter and define a check method inside security config to verify the route is accepted or not

Regex	Using static variables to contain regular expressions to check inputs	Keep it the same as this is an utility class
-------	---	--

In earlier stage, there were some problems and the improvements are made:

Table 60 Stamp coupling improvements

Related modules	Description	Improvements
CartService	addToCart and checkProductExistIncart are taking the whole User as parameter, but only need user email	Passing only String email instead of User user.
JwtService	generateToken takes the whole User userDetails but only use role and username attribute	Passing only String userRole and String username instead of User userDetails
ProductService	getWishList are taking the whole User but only user email	Passing only String userEmail instead

As for the rest of the system, modules reach the state of data coupling and functional cohesion

5.4 Design Principles

In the first draft of the design, there were couples of issues when using SOLID principles to qualify.

As for Single Responsibility Principle, all modules satisfy the standard as no cohesion problems are found.

When it comes to Open-Closed Principle, module SecurityConfig is having Common coupling issue that may seem violating the principle, when having another route in WHITE_LIST, developers have to modify the module. Module Regex is having Common coupling issue that violates the principle but as a configuration module. However, the classes are designed to be configurable and extensible through the use of Spring's configuration mechanisms, such as defining beans and using method overrides. Some

service modules, namely CartService, JwtService, ProductService, are having Stamp coupling issue, which may lead to violation of the principles if any changes to the structure of the class User are to be made. As for improvements, some directions are mentioned in Cohesion and Coupling review report, and with those alternations, these problems are mostly resolved.

In terms of Liskov Substitution Principle, ome custom base classes such as BaseEntity, BaseResponse are defined for generalization and its inherited classes can fully be substitutable for its parent class. All the Entity classes do not conflict with BaseEntity, as for response, at this stage, there has not been anything to judge. Inheritance trees are all 2-generation and have not been found with conflicts.

Interfaces are defined as classes named with the suffix of 'Repository', each serves a certain functionality without too many methods in one. There have been interfaces for managing Order, Product, ProductCart, User, WishList and Review(the last two are extended functionality that our group may design and implement). Therefore, there is no issue with Interface Segregation Principle.

Last but not least, prior to the current design, the structure of communication goes from: Controller – Service – Repository, high level – low level – repository, and current implementation shows that the controller depends directly on service, which violates the principle. The improvement direction is to generalize an interface for controller – service, let the service implements the interface and the controller will depend on the interface. After modifications, the problems are addressed.

5.5 Design Patterns

5.5.1 Singleton Pattern

The Singleton pattern ensures that a class has only one instance and provides a global point of access to it. In Spring, beans defined with the '@Bean' annotation are singletons by default.

Example:

@Bean

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

5.5.2 Dependency Injection

Dependency Injection (DI) is a fundamental aspect of Spring. It allows the creation of dependent objects outside of a class and provides those objects to a class in various ways.

Example:

```
@RestController
@RequiredArgsConstructor
@RequestMapping("/api/user")
public class UserController {
    final IUserService userService;
    final HttpServletRequest request;
    final ModelMapper modelMapper;
}
```

5.5.3 Factory Method Pattern

The Factory Method pattern defines an interface for creating an object but allows subclasses to alter the type of objects that will be created. In Spring, the `@Bean` annotation can be seen as a factory method.

Example:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http, JwtFilter jwtFilter)
throws Exception {
    // Configuration code
    return http.build();
}
```

5.5.4 Template Method Pattern

The Template Method pattern defines the skeleton of an algorithm in a method, deferring some steps to subclasses. Spring Security's configuration classes often use this pattern.

Example:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
```

```

@Bean

public SecurityFilterChain securityFilterChain(HttpSecurity http, JwtFilter jwtFilter)
throws Exception {

    http

        .cors(AbstractHttpConfigurer::disable)

        .csrf(AbstractHttpConfigurer::disable)

        .authorizeHttpRequests(auth -> {

            auth.requestMatchers("/api/auth/**").permitAll();

            auth.anyRequest().authenticated();

        })

        .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))

        .exceptionHandling(exception -> exception

            .authenticationEntryPoint((request, response, authException) ->
response.sendError(HttpServletResponse.SC_UNAUTHORIZED))

        )

        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();

}
}

```

5.5.5 Adapter Pattern

The Adapter pattern allows incompatible interfaces to work together. In Spring, the ModelMapper is used to map between different object models.

Example:

```

@GetMapping("/info")

public ResponseEntity<?> getInfo() {

    return                BaseResponse.ok(modelMapper.map(request.getAttribute("user"),
UserDto.class));

}

```

5.5.6 Builder Pattern

The Builder pattern is used to construct complex objects step by step. The `HttpSecurity` configuration in Spring Security can be seen as an example of the Builder pattern.

Example:

http

```
.cors(AbstractHttpConfigurer::disable)

.csrf(AbstractHttpConfigurer::disable)

.authorizeHttpRequests(auth -> {
    auth.requestMatchers("/api/auth/**").permitAll();
    auth.anyRequest().authenticated();
})

.sessionManagement(session ->
    session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))

.exceptionHandling(exception -> exception
    .authenticationEntryPoint((request, response, authException) ->
        response.sendError(HttpServletResponse.SC_UNAUTHORIZED))
    )

.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
```

5.5.7 Strategy Pattern

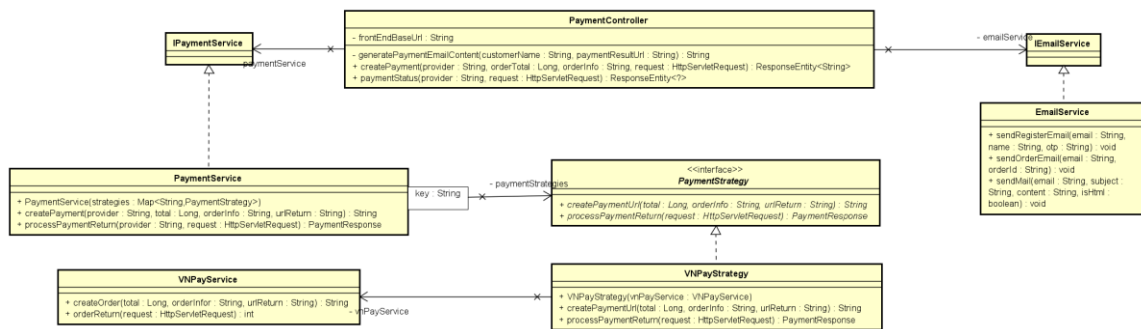


Figure 10 Strategy pattern in payment handling

Payment handling is implemented with strategy pattern ensuring expandability and flexibility of the payment methods integrated into the system.